
Causal Attention with Lookahead Keys

Zhuoqing Song^{1†} Peng Sun² Huizhuo Yuan² Quanquan Gu²

Abstract

In standard causal attention, each token’s query, key, and value (QKV) are static and encode only preceding context. We introduce CAuSal aTtention with Lookahead kEys (CASTLE), an attention mechanism that continually updates each token’s keys as the context unfolds. We term these updated keys lookahead keys because they belong to earlier positions yet integrate information from tokens that appear later relative to those positions, while strictly preserving the autoregressive property. Although the mechanism appears sequential, we derive a mathematical equivalence that avoids explicitly materializing lookahead keys at each position and enables efficient parallel training. On language modeling benchmarks, CASTLE consistently outperforms standard causal attention across model scales, reducing validation perplexity and improving average performance on a range of downstream tasks.

1. Introduction

Causal attention (Vaswani et al., 2017) is a cornerstone of autoregressive sequence modeling, allowing each token to condition on its past while preserving the autoregressive structure that underpins language generation. Building on this mechanism, large language models (LLMs) have transformed natural language processing by scaling up the model size and the number of tokens trained (Radford et al., 2019; Brown et al., 2020; Kaplan et al., 2020). While this trend has delivered impressive capabilities, it increasingly runs up against a practical bottleneck, that is high-quality tokens. This reality makes it imperative to improve the attention layer itself and to develop model architectures that are more token-efficient, delivering stronger performance under fixed training-token budgets.

[†]Work done during an internship at ByteDance Seed.

¹Department of Electrical and Computer Engineering, Princeton University ²ByteDance Seed. Correspondence to: Quanquan Gu <quanquan.gu@bytedance.com>.

In standard causal attention, each token’s query, key, and value (QKV) are computed from that token’s representation and remain fixed; they cannot incorporate information from subsequent tokens. As a result, a token’s QKV can encode only its preceding context. Recent work shows that such causal mask, which blocks each token’s access to its future information, limits models’ ability to capture global context, and impairs natural language understanding (Li & Li, 2024; Du et al., 2022; Springer et al., 2025; Xu et al., 2024; Zhang et al., 2025). Here are some illustrations that provide intuition for the limitations of causal masking. Garden-path sentences (Pritchett, 1987) are structurally ambiguous, often inducing an incorrect initial parse. For example, “the old man the boat”. Because garden-path sentences’ correct interpretation typically depends on information that appears later in the sentence, the causal mask restricting tokens to past context can cause models to struggle in resolving such ambiguities effectively (Amouyal et al., 2025). In many tasks, the question/focus appears at the end of the input. Without access to future context, earlier tokens cannot encode the relevant information needed to anticipate the question/focus. As a result, early token representations may fail to capture important cues and global context dependencies (Xu et al., 2024).

To address the shortcomings of standard causal attention in pretraining, we propose a novel attention mechanism, CAuSal aTtention with Lookahead kEys (CASTLE). In this approach, when generating the $(t + 1)$ -th token, we update keys of all preceding tokens so that keys of token s ($1 \leq s \leq t$) are able to additionally encode information from token $s + 1$ to t . These keys are called lookahead keys. This design preserves the autoregressive structure while allowing the keys to evolve as the context unfolds. In Figure 1, we give an illustration of receptive fields of the keys in CASTLE. Although the mechanism appears recurrent, we establish a mathematical equivalence that avoids explicitly materializing the lookahead keys and enables efficient parallel training. We evaluate our approach on language modeling across multiple model scales. Experimental results show that CASTLE consistently outperforms standard causal attention in terms of validation perplexity and downstream task performance. We also introduce a variant, CASTLE-SWL, which applies sliding windows to lookahead keys. CASTLE-SWL has the same complexities as CASTLE in

training and inference and preserves the performance benefits of CASTLE while further improving efficiency and saving KV-cache in practice.

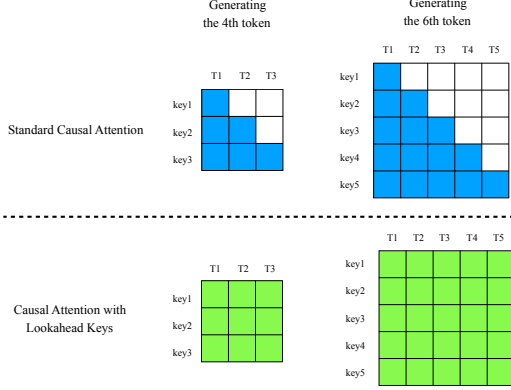


Figure 1. Receptive fields of keys in standard causal attention and CASTLE when generating the 4th and 6th tokens. In standard causal attention, each key i is static and only encodes information of tokens $\{T_1, \dots, T_i\}$. In CASTLE, keys are updated at each prediction step, so when generating token $t + 1$, every key $i \leq t$ can encode information from $\{T_1, \dots, T_t\}$.

The remainder of this paper is organized as follows. We discuss related work in Section 1.1. In Section 1.2, we elaborate on our motivations. In Section 1.3, we formally define CASTLE in its recurrent form. Section 1.4 proves an equivalent parallel formulation of CASTLE and develops efficient parallel training algorithms. Section 1.5 introduces efficient inference algorithms together with the counterpart of the KV cache in CASTLE. Finally, Section 2 presents empirical results demonstrating the effectiveness of CASTLE across diverse tasks and model scales.

1.1. Related Work

Several studies have observed that the causal mask, by preventing tokens from accessing future information, can hinder a model’s ability to capture global context and degrade its natural language understanding (Li & Li, 2024; Du et al., 2022; Springer et al., 2025; Xu et al., 2024; Zhang et al., 2025).

Much effort has been made to overcome this shortcoming in sentence embedding. BeLLM (Backward Dependency Enhanced LLM) (Li & Li, 2024) modifies decoder layers to enable sentence embeddings to integrate both past and future information. This yields substantial improvements on semantic textual similarity and downstream tasks. Echo Embeddings (Springer et al., 2025) duplicate the input sequence and extract embeddings from the second occurrence, letting early tokens attend to later ones without modifying the architecture. Similarly, Re-Reading (RE2) (Xu et al., 2024) prompts models to process inputs twice, so the second pass captures global information obtained in the first.

These methods improve embedding quality and reasoning, but their benefits in large-scale pretraining remain unclear. Further related work can be found in Appendix A.

1.2. Motivations

To explain our motivations, we begin with the recurrent form of standard causal attention. Consider the case when generating the $(t + 1)$ -th token. Given contextualized representations $\mathbf{X}^t = (\mathbf{x}_1^\top, \dots, \mathbf{x}_t^\top)^\top \in \mathbb{R}^{t \times d_{\text{model}}}$, where the s -th row $\mathbf{x}_s$ is the representation of token s . Unless otherwise specified, all vectors in this paper are treated as row vectors rather than column vectors.

The query, key and value of token s are $\mathbf{q}_s = \mathbf{x}_s \mathbf{W}_Q$, $\mathbf{k}_s = \mathbf{x}_s \mathbf{W}_K$, $\mathbf{v}_s = \mathbf{x}_s \mathbf{W}_V$. We also denote $\mathbf{K}_t = \mathbf{X}^t \mathbf{W}_K$ and $\mathbf{V}_t = \mathbf{X}^t \mathbf{W}_V$. Then, the standard causal attention follows the recurrent form

$$\begin{aligned} & \text{causal-attention}(\mathbf{X}^t) \\ &= \text{softmax} \left(\frac{\mathbf{q}_t \mathbf{K}_t^\top}{\sqrt{d}} \right) \mathbf{V}_t \\ &= \frac{\sum_{s=1}^t \exp(\mathbf{q}_t \mathbf{k}_s^\top / \sqrt{d}) \mathbf{v}_s}{\sum_{s=1}^t \exp(\mathbf{q}_t \mathbf{k}_s^\top / \sqrt{d})} \in \mathbb{R}^{1 \times d}. \end{aligned} \quad (1)$$

Due to the autoregressive structure, each $\mathbf{x}_s$ only encodes information from token 1 to s . Thus, when generating token $t + 1$ with $t + 1 > s$, each $\mathbf{k}_s$ only contains information from token 1 to s without containing information from token $s + 1$ to t . This can impair models’ ability to understand language, yield high-quality text embeddings, and capture global context, as discussed in the introduction.

This motivates us to propose a novel attention mechanism, causal attention with lookahead keys (CASTLE), i.e., when generating the $(t + 1)$ -th token, we first update keys $\mathbf{k}_s$ of preceding tokens s with $s < t + 1$ to additionally incorporate information from token $s + 1$ to t . We refer to these as *lookahead keys* because their representations renew with the growing context. In this way, lookahead keys may lead to more accurate attention scores while preserving the autoregressive property. The formal definition of lookahead keys is given in Section 1.3. It also has an equivalent definition so that for each incoming token, existing lookahead keys are updated by a rank-1 update as in (12).

We remark that this mechanism does not violate autoregressiveness. When generating token $t + 1$, we update keys of each preceding token $s < t + 1$ with information from token $s + 1$ to t . Thus, all keys only contain information from token 1 to t . Queries and values are naturally only containing information from tokens up to t . No future information from tokens $k > t$ is used. Thus, the model maintains the autoregressive property.

The reason that we use lookahead keys instead of looka-

head queries parallels the reason why key-value (KV) pairs are cached instead of queries (Q). Each q_s is only used once, namely when generating token $s + 1$. Because we are designing an autoregressive model, past queries cannot be modified after generation, making it meaningless to update q_s . In contrast, k_s is multiplied by the queries q_t of all subsequent tokens $t \geq s$. Keeping k_s updated therefore can benefit all later tokens by possibly producing more accurate attention scores. Updating the values v_s similarly to lookahead keys could be beneficial, but designing an efficient algorithm remains future work.

Further details of our design are presented in Section 1.3.

1.3. Mathematical Definition in Recurrent Form

We give mathematical definitions of CASTLE in this section. Let L denote sequence length and d_{model} , d denote the hidden dimension and head dimension, respectively.

Throughout Section 1.3, we fix $t \in \{1, 2, \dots, L\}$ and consider the setting where t tokens have been generated and the model is generating the $(t + 1)$ -th token. Denote the input contextualized representations $\mathbf{X}^t = (\mathbf{x}_1^\top, \dots, \mathbf{x}_t^\top)^\top \in \mathbb{R}^{t \times d_{\text{model}}}$, where $\mathbf{x}_s$ is the representation of token s .

Utilizing lookahead keys lies at the core of CASTLE. However, the way a model learns to encode information into keys of token s from past tokens ($k \leq s$) may differ from how it encodes information from subsequent tokens (tokens $s < k < t + 1$) when generating the $(t + 1)$ -th token. To address this, we adopt a hybrid design by partitioning the keys into two equal groups. One half is the same as in standard causal which we call *causal keys*. The remaining half renews as the context progresses which we call *lookahead keys*.

For each preceding token s ($1 \leq s < t + 1$), the causal key of token s is a projection of $\mathbf{x}_s$, while the lookahead key of token s contains information from representations $\{\mathbf{x}_{s+1}, \dots, \mathbf{x}_t\}$. The receptive fields of causal keys and lookahead keys are illustrated in Figure 13.

We first project $\mathbf{X}^t$ into key and value matrices $\mathbf{K}_t^U, \mathbf{V}_t^U, \mathbf{K}_t^C, \mathbf{V}_t^C \in \mathbb{R}^{t \times d}$ by $\mathbf{K}_t^U = \mathbf{X}^t \mathbf{W}_K^U, \mathbf{V}_t^U = \mathbf{X}^t \mathbf{W}_V^U, \mathbf{K}_t^C = \mathbf{X}^t \mathbf{W}_K^C, \mathbf{V}_t^C = \mathbf{X}^t \mathbf{W}_V^C$ as well as query matrix $\mathbf{Q}_t^U = \mathbf{X}^t \mathbf{W}_Q^U \in \mathbb{R}^{t \times d}$ and query vector $\mathbf{q}_t^C = \mathbf{x}_t \mathbf{W}_Q^C \in \mathbb{R}^{1 \times d}$. Here, $\mathbf{W}_Q^U, \mathbf{W}_K^U, \mathbf{W}_V^U, \mathbf{W}_Q^C, \mathbf{W}_K^C, \mathbf{W}_V^C \in \mathbb{R}^{d_{\text{model}} \times d}$ are learnable matrices.

The matrices $\mathbf{K}_t^U, \mathbf{V}_t^U, \mathbf{Q}_t^U$ are used to generate the lookahead key $\mathbf{U}^t$. Then, the causal key $\mathbf{K}_t^C$ and the lookahead key $\mathbf{U}^t$ are multiplied by the query vector $\mathbf{q}_t^C$ to get the attention scores. Then, $\mathbf{V}_t^C$ are multiplied by the attention weights to get the output. Before we elaborate on details in the definition of CASTLE, we first give formal definitions of causal keys and lookahead keys.

Causal Keys. The causal keys in CASTLE are defined similarly to the keys in standard causal attention. More specifically, causal keys are defined as $\mathbf{K}_t^C = (k_1^C, \dots, k_t^C)^\top = \mathbf{X}^t \mathbf{W}_K^C \in \mathbb{R}^{t \times d}$. The s -th row k_s of $\mathbf{K}_t^C$ satisfying $k_s = \mathbf{x}_s \mathbf{W}_K^C$ is the causal key of token s . Causal keys are static, i.e., the s -th rows of $\mathbf{K}_t^C$ and $\mathbf{K}_{t'}^C$ are equal to each other whenever $t, t' \geq s$.

Lookahead Keys. We utilize a structure similar to the attention mechanism to define lookahead keys. An illustration for lookahead keys can be found in Figure 2.

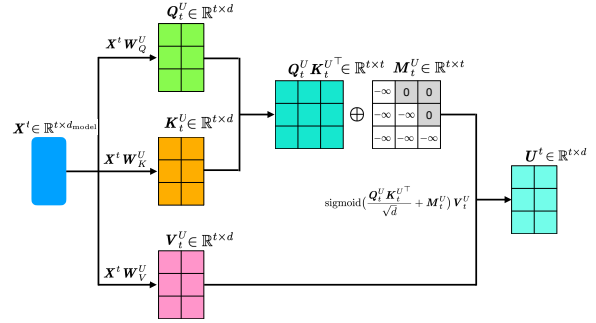


Figure 2. Illustration for the definition of lookahead keys in (2) when generating the 4-th token. Here d_{model} and d denote the hidden dimension and head dimension, respectively. In this figure, we set $t = 3$ and $d = 2$.

More specifically, the lookahead keys are defined as

$$\begin{aligned} \mathbf{U}^t &= (\mathbf{u}_1^\top, \dots, \mathbf{u}_t^\top)^\top \\ &= \sigma\left(\frac{\mathbf{Q}_t^U \mathbf{K}_t^{U\top}}{\sqrt{d}} + \mathbf{M}_t^U\right) \mathbf{V}_t^U \in \mathbb{R}^{t \times d}, \end{aligned} \quad (2)$$

where $\mathbf{M}_t^U \in \mathbb{R}^{t \times t}$ is a mask matrix with

$$[\mathbf{M}_t^U]_{ij} = \begin{cases} 0, & i < j, \\ -\infty, & \text{otherwise.} \end{cases} \quad (3)$$

The s -th row $\mathbf{u}_s^t$ of $\mathbf{U}^t$ is the lookahead key of token s . We remark that in $\mathbf{u}_s^t$, the superscript t indicates that $\mathbf{u}_s^t$ is defined when t tokens have already been generated and we are about to generate the $(t + 1)$ -th token, while the subscript s indicates $\mathbf{u}_s^t$ is the s -th row of $\mathbf{U}^t$.

The definition of $\mathbf{M}_t^U$ guarantees that the lookahead key of token s , $\mathbf{u}_s^t$, is exposed to information from $\{\mathbf{x}_{s+1}, \dots, \mathbf{x}_t\}$. Since $\mathbf{u}_s^t$ keeps renewing as the context goes, it is natural that $\mathbf{u}_s^t \neq \mathbf{u}_s^{t+1}$.

We have the following remarks regarding the definition of lookahead keys in (2).

• **Why are we using sigmoid?** The σ function is used in (2) instead of softmax due to the consideration that when generating token $t + 1$, for token s with $s < t + 1$, synthesizing

information contained in tokens $s + 1$ to t should not be compulsory. However, since the probabilities in softmax sum up to 1, which forces u_s^t to incorporate information from tokens $s + 1$ to t and is not desired.

- **Lookahead keys U^t maintains autoregressive property.** First, CASTLE is defined in a recurrent form which is naturally autoregressive. Second, when generating the $(t + 1)$ -th token, each u_s^t is only exposed to information from representations of tokens $s + 1$ to t as in (2). No information from tokens which are not yet generated is exposed.

- **Lookahead keys U^t only occur in CASTLE’s recurrent form definition and inference, but cannot be materialized in parallel training.** Since u_s^t and u_{s+1}^t may vary, this prevents us from materializing U^t for each t . The computation cost in (2) is $O(t^2d)$. If we materialize all U^t in parallel, the computational cost is at least $\sum_{t=1}^L t^2d = O(L^3d)$ which makes training on large-scale datasets impractical. In Section 1.4, we will give an equivalent form which removes the need of materializing each U^t and enables efficient parallel training.

CASTLE in Recurrent Form. After defining causal keys and lookahead keys, we are ready to give the formula of CASTLE in recurrent form. To generate the $(t + 1)$ -th token, we utilize both the causal keys $K_t^C \in \mathbb{R}^{t \times d}$ and the lookahead keys U^t . An illustration of CASTLE in its recurrent form can be found in Figure 3. More specifically,

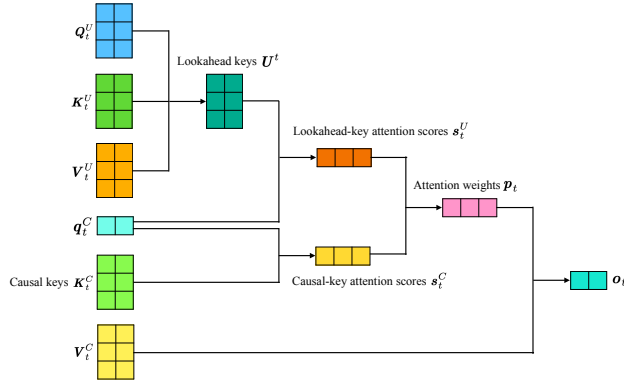


Figure 3. Illustration of CASTLE in recurrent form when generating the 4th token.

let the *causal-key attention scores* be

$$s_t^C = \frac{q_t^C K_t^{C\top}}{\sqrt{d}} \in \mathbb{R}^{1 \times t}. \quad (4)$$

Let the *lookahead-key attention scores* be

$$s_t^U = \frac{q_t^C U^{t\top}}{\sqrt{d}} \in \mathbb{R}^{1 \times t}. \quad (5)$$

Then, we define attention weights by combining the above

attention scores as follows

$$p_t = \text{softmax}(s_t^C - \text{SiLU}(s_t^U)) \in \mathbb{R}^{1 \times t}, \quad (6)$$

where $\text{SiLU}(x) = x \odot \sigma(x)$. Then, the output is calculated as

$$\text{attention}(X^t) = p_t V_t^C \in \mathbb{R}^{1 \times d}. \quad (7)$$

We remark that SiLU is applied in (6) because our ablation study indicates that it plays a crucial role in ensuring training stability. We give a comprehensive study of different variants of (6) and discuss why we use SiLU in Appendix B.4.3.

CASTLE-SWL in Recurrent Form. We propose a variant of CASTLE, termed CASTLE-SWL, where we apply Sliding Window to Lookahead keys. More specifically, denote sliding window size by W . When generating the $(t + 1)$ -th token, lookahead keys of token s ($s < t + 1$) have access to information from $\{x_k : s + 1 \leq k \leq \min\{s + W, t\}\}$. Formally, the definition of lookahead keys in CASTLE-SWL follows (2) with M_t^U defined as

$$[M_t^U]_{ij} = \begin{cases} 0, & i < j \leq \min(t, i + W), \\ -\infty, & \text{otherwise.} \end{cases} \quad (8)$$

An illustration for M_t^U applied to lookahead keys in CASTLE-SWL can be found in Figure 14. Sliding window modifies only lookahead keys while causal keys remains unchanged. Apart from changing the definition of M_t^U from (3) to (8), all other components of CASTLE remain the same, yielding the definition of CASTLE-SWL. The motivation for this design is that the semantic contribution of tokens typically decays as the context unfolds. Allowing lookahead keys to aggregate information from distant tokens may introduce noise rather than useful signal. In addition, although CASTLE-SWL has the same complexities with CASTLE in both training and inference as analyzed in Section 1.4 and 1.5, it can reduce FLOPs and further improves efficiency and saving UQ-KV cache in practice.

1.4. Efficient Parallel Training

In this section, we introduce our efficient parallel training algorithm. As discussed in Section 1.3, straightforwardly materializing each U^t in parallel incurs at least $O(L^3d)$ computational cost. Such complexity makes training on large-scale datasets infeasible. In addition, it is hard to derive an efficient parallel training algorithm based on the recurrent formulation introduced in Section 1.3 above. To address this, we derive an equivalent parallel formulation of CASTLE and CASTLE-SWL that eliminates the need to materialize lookahead keys. This formulation then serves as the basis for designing an efficient algorithm.

CASTLE and CASTLE-SWL in Parallel Form. Let $\text{attention}(\mathbf{X}^t) \in \mathbb{R}^{1 \times d}$ denote the output when generating the $(t+1)$ -th token as in (7). Then, given the inputs $\mathbf{X}^L \in \mathbb{R}^{L \times d_{\text{model}}}$, the concatenated outputs are denoted by

$$\text{Attention}(\mathbf{X}^L) = \begin{pmatrix} \text{attention}(\mathbf{X}^1) \\ \text{attention}(\mathbf{X}^2) \\ \vdots \\ \text{attention}(\mathbf{X}^L) \end{pmatrix} \in \mathbb{R}^{L \times d_{\text{model}}}. \quad (9)$$

The following theorem provides a unified parallel formulation for both CASTLE and CASTLE-SWL that is equivalent to their recurrent forms introduced in Section 1.3. Its proof is in Appendix C.

Theorem 1.1. Consider inputs $\mathbf{X}^L \in \mathbb{R}^{L \times d_{\text{model}}}$, where L is the sequence length and d_{model} is the hidden dimension. Let $\mathbf{Q}^U = \mathbf{X}^L \mathbf{W}_Q^U$, $\mathbf{K}^U = \mathbf{X}^L \mathbf{W}_K^U$, $\mathbf{V}^U = \mathbf{X}^L \mathbf{W}_V^U$, $\mathbf{Q}^C = \mathbf{X}^L \mathbf{W}_Q^C$, $\mathbf{K}^C = \mathbf{X}^L \mathbf{W}_K^C$, $\mathbf{V}^C = \mathbf{X}^L \mathbf{W}_V^C$. Define matrix $\mathbf{S}^U \in \mathbb{R}^{L \times L}$ as

$$\mathbf{S}^U = \left(\frac{\mathbf{Q}^C \mathbf{V}^{U\top}}{\sqrt{d}} \odot \widetilde{\mathbf{M}}^C \right) \left(\sigma \left(\frac{\mathbf{Q}^U \mathbf{K}^{U\top}}{\sqrt{d}} + \mathbf{M}^U \right) \right)^\top, \quad (10)$$

where σ is the sigmoid function. Then, the outputs $\text{Attention}(\mathbf{X}^L)$ as in (9) satisfies that

$$\begin{aligned} & \text{Attention}(\mathbf{X}^L) \\ &= \text{softmax} \left(\frac{\mathbf{Q}^C \mathbf{K}^{C\top}}{\sqrt{d}} + \mathbf{M}^C - \text{SiLU}(\mathbf{S}^U) \right) \mathbf{V}^C. \end{aligned} \quad (11)$$

Here, $\mathbf{M}^C$, $\widetilde{\mathbf{M}}^C$ are the causal masks which prevent tokens from attending to their future tokens, i.e., $\mathbf{M}_{ij}^C = 0$ if $i \geq j$ and $\mathbf{M}_{ij}^C = -\infty$ otherwise; $\widetilde{\mathbf{M}}_{ij}^C = 1$ if $i \geq j$ and $\widetilde{\mathbf{M}}_{ij}^C = 0$ otherwise. For CASTLE, $\mathbf{M}^U = \mathbf{M}_L^U \in \mathbb{R}^{L \times L}$ with $\mathbf{M}_L^U$ defined in (3), while for CASTLE-SWL, $\mathbf{M}^U = \mathbf{M}_L^U \in \mathbb{R}^{L \times L}$ with $\mathbf{M}_L^U$ defined in (8).

Efficient Parallel Training. Theorem 1.1 establishes the equivalence between the recurrent and parallel formulations for CASTLE and CASTLE-SWL. However, computing $\text{Attention}(\mathbf{X}^L)$ directly from Theorem 1.1 still requires $\Omega(L^3)$ complexity, since (10) involves matrix multiplications between L -by- L matrices.

To reduce this cost, notice that in (10), the term $(\mathbf{Q}^C \mathbf{V}^{U\top}) \odot \widetilde{\mathbf{M}}^C$ is a masked low-rank matrix because the matrix $\mathbf{Q}^C \mathbf{V}^{U\top}$ is of rank d which is typically much smaller than L . We design FlashAttention-style kernels utilizing this structure to further reduce computational complexities and avoid materializing $\mathbf{S}^U$.

Theorem 1.2. Given $\mathbf{X}^L$'s query, key and value matrices $\mathbf{Q}^U = \mathbf{X}^L \mathbf{W}_Q^U$, $\mathbf{K}^U = \mathbf{X}^L \mathbf{W}_K^U$, $\mathbf{V}^U = \mathbf{X}^L \mathbf{W}_V^U$,

$\mathbf{Q}^C = \mathbf{X}^L \mathbf{W}_Q^C$, $\mathbf{K}^C = \mathbf{X}^L \mathbf{W}_K^C$, $\mathbf{V}^C = \mathbf{X}^L \mathbf{W}_V^C$, for both CASTLE and CASTLE-SWL,

- FlashAttention-style implementation of forward pass is in Algorithm 1 which computes $\text{Attention}(\mathbf{X}^L)$ with computational complexity $O(L^2 d)$ and space complexity $O(Ld)$.
- FlashAttention-style implementation of backward pass is in Algorithm 3 which computes gradients with computational complexity $O(L^2 d)$ and space complexity $O(Ld)$.

Due to space limitation, more details of implementations for forward and backward pass are given in Appendix E.

1.5. Efficient Inference with UQ-KV Cache

In this section, we introduce the inference algorithm which has unified forms for CASTLE and CASTLE-SWL. A detailed description of our inference algorithm is provided in Appendix F. We first introduce the decoding algorithm. The decoding algorithm consists of the following 2 steps: *updating step* and *combining step*. Fix $t \in \{1, 2, \dots, L\}$, and consider generating the $(t+1)$ -th token.

Rank-1 Updating step. We generate lookahead keys $\mathbf{U}^t$ in the updating step. First, we compute $\mathbf{q}_t^U = \mathbf{x}_t \mathbf{W}_Q^U$, $\mathbf{k}_t^U = \mathbf{x}_t \mathbf{W}_K^U$ and $\mathbf{v}_t^U = \mathbf{x}_t \mathbf{W}_V^U$. Next, rather than computing $\mathbf{U}^t$ directly from (2), which requires $O(t^2 d)$ computation, we obtain $\mathbf{U}^t$ through rank-1 update from $\mathbf{U}^{t-1}$

$$\mathbf{U}^t = \left(\mathbf{U}^{t-1} + \sigma \left(\frac{\mathbf{Q}_{t-1}^U \mathbf{k}_t^{U\top}}{\sqrt{d}} + [\mathbf{M}_t^U]_{1:t-1,t} \right) \mathbf{v}_t^{U\top} \right) \mathbf{0}^{1 \times d} \quad (12)$$

where $[\mathbf{M}_t^U]_{1:t-1,t}$ is the t -th column of $\mathbf{M}_t^U$ removing the last entry. The proof of (12) is given in Appendix F. Next, we discuss the caching strategy and FLOPs of the updating step.

• **Caching in updating step.** First, it is obvious that we need to cache $\mathbf{U}^t$ so that we can implement the recursive formula (12). Second, we need to cache $\mathbf{Q}_t^U$ because $\mathbf{q}_s^U$ is used in any update from $\mathbf{U}^{t-1}$ to $\mathbf{U}^t$ with $s \leq t-1$. As $\mathbf{k}_t^U$ and $\mathbf{v}_t^U$ are only used in the update from $\mathbf{U}^{t-1}$ to $\mathbf{U}^t$ and never used again in update from $\mathbf{U}^{j-1}$ to $\mathbf{U}^j$ with $j \neq t$, we do not need to cache any other variables except $\mathbf{U}^t$ and $\mathbf{Q}_t^U$ for the updating step.

• **FLOPs in updating step.** With cached $\mathbf{U}^{t-1}$ and $\mathbf{Q}_{t-1}^U$, the updating formula (12) needs only $O(td)$ FLOPs. And computing $\mathbf{q}_t^U$, $\mathbf{k}_t^U$ and $\mathbf{v}_t^U$ needs $O(dd_{\text{model}})$ FLOPs, yielding total FLOPs of $O(dd_{\text{model}} + td)$.

Combining step. In the combining step, we compute $\mathbf{q}_t^C = \mathbf{x}_t \mathbf{W}_Q^C$, $\mathbf{k}_t^C = \mathbf{x}_t \mathbf{W}_K^C$ and $\mathbf{v}_t^C = \mathbf{x}_t \mathbf{W}_V^C$. Next, the attention outputs are then obtained by applying (4), (5), (6) and (7) with $O(td)$ FLOPs. At this stage, since we already cached $\mathbf{U}^t$ in the updating step, only $\mathbf{K}_t^C$ and $\mathbf{V}_t^C$ need to be stored.

UQ-KV cache. From the above analysis, the counterpart of the KV cache in CASTLE consists of U^t , Q_t^U , K_t^C , and V_t^C , which we collectively refer to as the *UQ-KV cache*. All other variables, including q_t^C , k_t^U , and v_t^U , can be safely disposed of after use.

For the prefilling stage, let the prompt length be L and inputs $X^L \in \mathbb{R}^{L \times d_{\text{model}}}$. We first compute $Q_L^U = X^L W_Q^U$, $K_L^U = X^L W_K^U$, $V_L^U = X^L W_V^U$, $Q_L^C = X^L W_Q^C$, $K_L^C = X^L W_K^C$, $V_L^C = X^L W_V^C$. Then, we apply the forward pass of the efficient parallel training algorithm (Algorithm 1) to get $\text{Attention}(X^L)$. For the UQ-KV cache, since we already have Q_L^U , K_L^C and V_L^C , we only need to obtain U^L . This can be done similarly to FlashAttention-2 (Dao, 2024) due to the similarity between (2) and standard causal attention. The complete prefilling procedure is given in Algorithm 5. The analysis above leads to the following theorem. Its proof and more detailed introduction of CASTLE’s inference is in Appendix F.

Theorem 1.3. *Given inputs $X^L \in \mathbb{R}^{L \times d_{\text{model}}}$, prefilling has computational complexity $O(Ldd_{\text{model}} + L^2d)$ and space complexity $O(Ld)$. During the decoding stage, when generating the t -th token, the computational complexity is $O(td + dd_{\text{model}})$ and the UQ-KV cache requires $O(td)$ memory.*

1.6. Multi-Head Causal Attention with Lookahead Keys

As in standard causal attention, we also utilize multi-head mechanism. Let n denote the number of heads. In each head i , when generating the t -th token, denote the output as in (7) by $\text{attention}_i(X^t) \in \mathbb{R}^{1 \times d}$. Then, the output of multi-head causal attention with lookahead keys (multi-head CASTLE) is calculated as

$$\begin{aligned} & \text{multi-head-attention}(X^t) \\ &= \text{Concat}_{1 \leq i \leq n} (\text{attention}_i(X^t)) W^O \in \mathbb{R}^{1 \times d}, \end{aligned} \quad (13)$$

where $W^O \in \mathbb{R}^{nd \times d_{\text{model}}}$ is a learnable matrix. The formula for forward pass in parallel training and more details of multi-head CASTLE are in Appendix D. Multi-head CASTLE-SWL shares the same formulation and parameter count as CASTLE, and is omitted here to avoid redundancy.

2. Experiments

2.1. Experimental Setup

We use the nanoGPT (Karpathy, 2023) code base. Our baseline follows the improved Transformer architecture with SwiGLU (Shazeer, 2020), Rotary Positional Embeddings (Su et al., 2024), and RMSNorm (Zhang & Sennrich, 2019) following LLaMA (Touvron et al., 2023). We train models on four scales from scratch: small (0.16B), medium (0.35B), large (0.75B) and XL (1.3B). The medium, large

and XL baseline models mirror the configuration of GPT-3 (Brown et al., 2020). For the small setting, we increase the number of heads and the hidden dimension relative to the original GPT-3 configuration to better align parameter counts between standard causal attention and CASTLE. To isolate the effect of the attention mechanism, we replace standard causal attention with CASTLE or CASTLE-SWL while keeping all other components unchanged for a fair comparison. We use the AdamW optimizer (Loshchilov & Hutter, 2019) and follow the training recipe of (Dao & Gu, 2024). We train models on FineWeb-Edu dataset (Lozhkov et al.) for 50B tokens. Further details of experimental setup can be found in Appendix B.1.

2.2. Training & Validation Loss

We report training and validation loss and perplexity in Table 1. Training loss and validation loss curves are shown in Figure 4, Figure 9, Figure 10 and Figure 11. CASTLE and CASTLE-SWL consistently outperform the baselines in both training and validation loss across all model scales.

Table 1. Training and validation loss and perplexity for models with CASTLE, CASTLE-SWL and standard causal attention. We use “S”, “M”, “L”, “XL” to denote model scales. Each model is trained on FineWeb-Edu for 50B tokens. The best and second best results (when showing clear improvements upon baselines) are highlighted in bold and underline, respectively.

	n_{params}	Train		Eval	
		Loss	PPL	Loss	PPL
Baseline-S	160M	2.795	16.364	2.798	16.411
CASTLE-S	160M	2.789	16.259	2.792	16.315
CASTLE-SWL-S	160M	2.786	16.213	2.790	16.273
Baseline-M	353M	2.641	14.030	2.639	14.004
CASTLE-M	351M	2.616	13.684	2.615	13.665
CASTLE-SWL-M	351M	<u>2.618</u>	<u>13.708</u>	<u>2.615</u>	<u>13.670</u>
Baseline-L	756M	2.513	12.346	2.507	12.269
CASTLE-L	753M	<u>2.476</u>	<u>11.897</u>	<u>2.472</u>	<u>11.840</u>
CASTLE-SWL-L	753M	2.476	11.890	2.471	11.832
Baseline-XL	1.310B	2.430	11.360	2.426	11.309
CASTLE-XL	1.304B	2.401	11.031	<u>2.391</u>	<u>10.922</u>
CASTLE-SWL-XL	1.304B	<u>2.401</u>	<u>11.036</u>	2.389	10.900

Specifically, after training 50B tokens, CASTLE outperforms baselines across all model scales and achieves validation losses that are 0.0059, 0.0245, 0.0356, and 0.0348 lower than the baseline for the small, medium, large, and XL models, respectively. CASTLE-SWL matches CASTLE’s performance and its validation loss outperforms baselines by 0.0084, 0.0241, 0.0366, 0.0369, respectively. The performance gains are particularly significant in medium, large, and XL models. Furthermore, as in Table 4, both CASTLE and CASTLE-SWL have the same or fewer parameters than the baseline counterparts. This further underscores the effectiveness of CASTLE(-SWL) in improving performance.

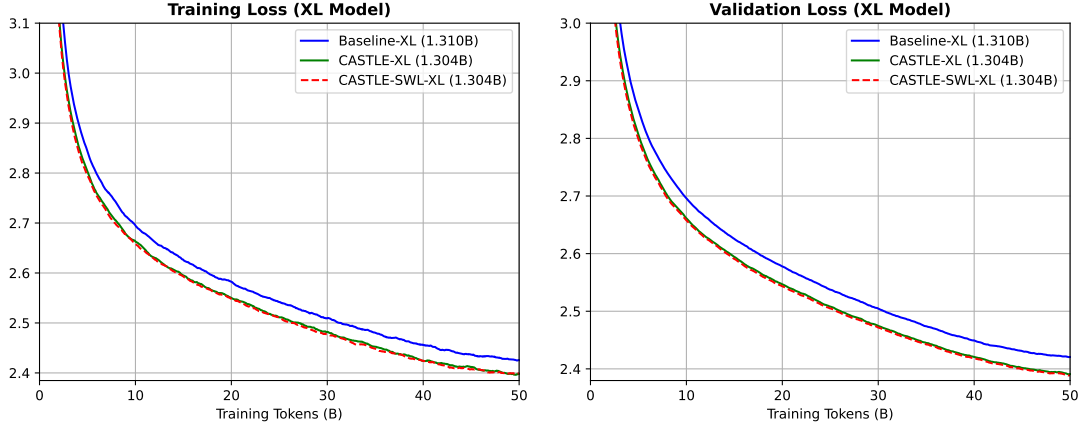


Figure 4. Training and validation loss curves of XL models. Training loss curve is smoothened with a moving window of 2000 training steps. Validation loss is evaluated every 100 training steps on 40M tokens, and its curve is smoothened by a moving window of 20 evaluation intervals. Loss curves for the small, medium and large models can be found in Figure 9, Figure 10 and Figure 11 of Appendix B.5. After 50B training tokens, CASTLE-XL achieves a 0.0294 lower training loss and a 0.0348 lower validation loss compared to Baseline-XL, while CASTLE-SWL-XL achieves a 0.0290 lower training loss and a 0.0369 lower validation loss compared to Baseline-XL.

Table 2. Evaluation results (5-shot) for downstream tasks of different model scales. The significantly higher accuracy values are shown in bold. All values denote accuracy in percentage (%). Each model is pretrained on FineWeb-Edu for 50B tokens. Hella.=HellaSwag, Wino.=Winogrande. 0-shot results can be found in Table 5.

Model Name	ARC-C	ARC-E	BoolQ	Hella.	OBQA	PIQA	SciQA	Wino.	Avg.
Baseline-S	25.68	54.97	56.09	33.81	28.20	63.98	84.90	52.57	50.02
CASTLE-S	<u>26.02</u>	54.25	57.13	<u>35.24</u>	<u>29.80</u>	<u>64.53</u>	<u>85.20</u>	50.99	<u>50.40</u>
CASTLE-SWL-S	27.39	56.19	<u>56.09</u>	35.46	30.20	64.85	86.50	<u>51.22</u>	50.99
Baseline-M	31.06	62.46	48.47	42.83	<u>33.00</u>	68.39	90.30	51.78	53.54
CASTLE-M	<u>32.17</u>	64.06	54.62	<u>43.47</u>	33.80	69.48	90.30	<u>52.49</u>	<u>55.05</u>
CASTLE-SWL-M	32.85	<u>63.85</u>	<u>54.19</u>	44.18	33.80	<u>69.04</u>	<u>90.20</u>	53.43	55.19
Baseline-L	33.36	63.64	<u>59.24</u>	46.16	33.40	<u>69.53</u>	90.40	54.06	56.22
CASTLE-L	37.37	67.89	50.95	<u>47.71</u>	34.20	70.18	91.60	54.06	<u>56.74</u>
CASTLE-SWL-L	<u>36.26</u>	<u>65.53</u>	60.58	48.55	34.80	69.10	92.00	<u>53.67</u>	57.56
Baseline-XL	35.58	65.78	61.07	50.84	36.20	<u>71.27</u>	91.40	52.72	58.11
CASTLE-XL	39.08	70.24	62.60	51.63	37.40	71.00	93.70	58.33	60.50
CASTLE-SWL-XL	<u>38.99</u>	<u>70.08</u>	<u>61.74</u>	52.35	<u>37.20</u>	72.52	94.00	<u>56.75</u>	<u>60.45</u>

2.3. Downstream Evaluation

We evaluate CASTLE and CASTLE-SWL on a diverse set of downstream benchmarks, including ARC (Yadav et al., 2019), BoolQ (Clark et al., 2019), HellaSwag (Zellers et al., 2019), OBQA (Mihaylov et al., 2018), PIQA (Bisk et al., 2020), SciQA (Auer et al., 2023), Winogrande (Sakaguchi et al., 2021) using lm-evaluation-harness (Gao et al., 2024). We report normalized accuracy for ARC-Challenge, HellaSwag, OBQA, and PIQA, and standard accuracy for the other benchmarks. Results are reported in Table 5 (0-shot) and Table 2 (5-shot). Across all model scales and evaluation settings, both CASTLE and CASTLE-SWL consistently outperform the baselines in average downstream accuracy under both 0-shot and 5-shot settings. In addition, for 5-shot evaluation, CASTLE-SWL-XL consistently improves over

Baseline-XL across all evaluated tasks; for 0-shot evaluation, CASTLE-SWL-XL improves over Baseline-XL on ARC-C, ARC-E, HellaSwag, OBQA, SciQA, Winogrande; for 5-shot evaluation, CASTLE-SWL improves upon baseline models on ARC-C, ARC-E, HellaSwag, OBQA across all model sizes.

3. Long-Context Evaluation

To evaluate models’ capabilities on long-context tasks, we train XL-size models on the LongCrawl64 dataset (Buckman) with sequence length 16K. Further training details are in Appendix B.3. For clarity, we refer to these models as CASTLE-SWL-XL-16K and Baseline-XL-16K, respectively.

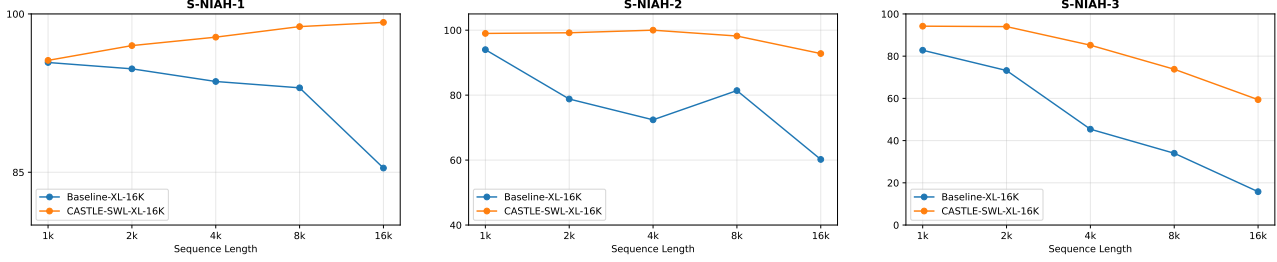


Figure 5. Results on the RULER S-NIAH tasks across different sequence lengths. CASTLE-SWL consistently outperforms the baseline model by a large margin across all tasks and sequence lengths.

3.1. Per-Position Loss

Following (Ye et al., 2025; Lin et al., 2025; Yang et al., 2025; Guo et al., 2026), we report models’ validation loss at each sequence position. The results are shown in Figure 6. As shown, CASTLE-SWL-XL-16K consistently achieves lower loss than the Baseline-XL-16K baseline across almost all token positions, indicating improved token-level prediction quality throughout the sequence.

On average of 16K positions, CASTLE-SWL-XL-16K achieves 0.074 lower validation loss than the Baseline-XL-16K model. The performance gap becomes even more pronounced as the token position increases.

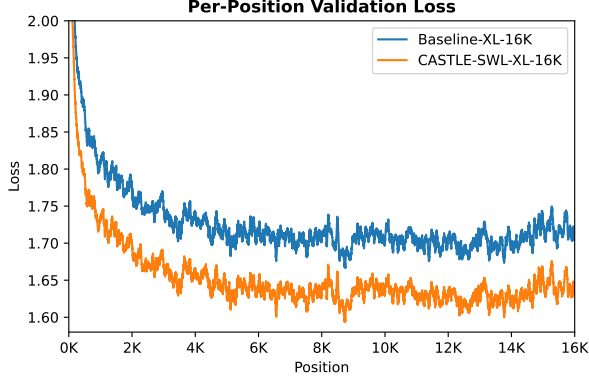


Figure 6. Per-position validation loss. Since per-position loss curve is not smooth, we use a moving average window of 50 tokens.

3.2. Needle-In-A-Haystack

We evaluate the models’ performance on the RULER S-NIAH tasks (Hsieh et al.). Table 3 and Figure 5 report results across different sequence lengths. CASTLE-SWL-XL-16K demonstrates consistently stronger retrieval capabilities than Baseline-XL-16K across all three S-NIAH settings and all sequence lengths, with particularly strong gains at longer sequence lengths. While the baseline performance degrades noticeably as sequence length increases, CASTLE maintains higher and more stable accuracy, demonstrating improved

long-context retrieval capability.

Table 3. Accuracy on the RULER S-NIAH tasks across different sequence lengths. Significantly higher accuracy is in bold.

	S-NIAH-1				
	1K	2K	4K	8K	16K
Baseline-XL-16K	95.4	94.8	93.6	93.0	85.4
CASTLE-SWL-XL-16K	95.6	97.0	97.8	98.8	99.2

	S-NIAH-2				
	1K	2K	4K	8K	16K
Baseline-XL-16K	94.0	78.8	72.4	81.4	60.2
CASTLE-SWL-XL-16K	99.0	99.2	100.0	98.2	92.8

	S-NIAH-3				
	1K	2K	4K	8K	16K
Baseline-XL-16K	82.8	73.2	45.4	34.0	15.8
CASTLE-SWL-XL-16K	94.2	94.0	85.2	73.8	59.4

4. Conclusion

We introduced CAuSal aTtention with Lookahead kEys (CASTLE), a novel attention mechanism that continually updates keys as the context evolves. This design allows each key representation to incorporate more recent information at every prediction step while strictly preserving the autoregressive property. Although CASTLE is defined recurrently, we derived a mathematical equivalence that eliminates the need to explicitly materialize lookahead keys at each position, enabling efficient parallel training. Experimental results on language modeling demonstrate that CASTLE consistently outperforms standard causal attention, achieving lower perplexity and stronger downstream performance.

Impact Statement

Our research focuses on developing a new attention mechanism for large language models. There are many potential societal consequences of our work, none of which we feel must be specifically highlighted here.

References

- Amouyal, S. J., Meltzer-Asscher, A., and Berant, J. When the LM misunderstood the human chuckled: Analyzing garden path effects in humans and language models. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 8235–8253, 2025.
- Auer, S., Barone, D. A., Bartz, C., Cortes, E. G., Jaradeh, M. Y., Karras, O., Koubarakis, M., Mouromtsev, D., Pliukhin, D., Radyush, D., et al. The sciqa scientific question answering benchmark for scholarly knowledge. *Scientific Reports*, 13(1):7240, 2023.
- Bisk, Y., Zellers, R., Gao, J., Choi, Y., et al. Piqa: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI conference on artificial intelligence*, volume 34, pp. 7432–7439, 2020.
- Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33: 1877–1901, 2020.
- Buckman, J. Longcrawl64: A long-context natural-language dataset, 2024. URL <https://manifestai.com/articles/longcrawl64>.
- Clark, C., Lee, K., Chang, M.-W., Kwiatkowski, T., Collins, M., and Toutanova, K. Boolq: Exploring the surprising difficulty of natural yes/no questions. In *Proceedings of the 2019 conference of the north American chapter of the association for computational linguistics: Human language technologies, volume 1 (long and short papers)*, pp. 2924–2936, 2019.
- Dao, T. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *International Conference on Learning Representations*, 2024.
- Dao, T. and Gu, A. Transformers are ssms: Generalized models and efficient algorithms through structured state space duality. In *International Conference on Machine Learning*, pp. 10041–10071. PMLR, 2024.
- Dao, T., Fu, D., Ermon, S., Rudra, A., and Ré, C. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in neural information processing systems*, 35:16344–16359, 2022.
- Du, Z., Qian, Y., Liu, X., Ding, M., Qiu, J., Yang, Z., and Tang, J. Glm: General language model pretraining with autoregressive blank infilling. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 320–335, 2022.
- Ewer, E., Chae, D., Zeng, T., Kim, J., and Lee, K. Entp: Encoder-only next token prediction. *arXiv preprint arXiv:2410.01600*, 2024.
- Gao, L., Tow, J., Abbasi, B., Biderman, S., Black, S., DiPofi, A., Foster, C., Golding, L., Hsu, J., Le Noac’h, A., Li, H., McDonell, K., Muennighoff, N., Ociepa, C., Phang, J., Reynolds, L., Schoelkopf, H., Skowron, A., Sutawika, L., Tang, E., Thite, A., Wang, B., Wang, K., and Zou, A. The language model evaluation harness, 07 2024. URL <https://zenodo.org/records/12608602>.
- Guo, H., Yang, S., Goel, T., Xing, E. P., Dao, T., and Kim, Y. Log-linear attention. In *International Conference on Learning Representations*, 2026.
- Hsieh, C.-P., Sun, S., Krizan, S., Acharya, S., Rekesh, D., Jia, F., and Ginsburg, B. RULER: What’s the real context size of your long-context language models? In *First Conference on Language Modeling*.
- Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., and Amodei, D. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.
- Karpathy, A. nanogpt: The simplest, fastest repository for training/finetuning medium-sized gpts. URL <https://github.com/karpathy/nanoGPT>, 2023.
- Leviathan, Y., Kalman, M., and Matias, Y. Selective attention improves transformer. In *International Conference on Learning Representations*, 2025.
- Li, X. and Li, J. Bellm: Backward dependency enhanced large language model for sentence embeddings. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pp. 792–804, 2024.
- Lin, Z., Nikishin, E., He, X., and Courville, A. Forgetting transformer: Softmax attention with a forget gate. In *International Conference on Learning Representations*, 2025.
- Loshchilov, I. and Hutter, F. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.
- Lozhkov, A., Allal, L. B., von Werra, L., and Wolf, T. Fineweb-edu: the finest collection of educational content, 2024. URL <https://huggingface.co/datasets/HuggingFaceFW/fineweb-edu>.
- Mihaylov, T., Clark, P., Khot, T., and Sabharwal, A. Can a suit of armor conduct electricity? a new dataset for open

- book question answering. In *Proceedings of the 2018 conference on empirical methods in natural language processing*, pp. 2381–2391, 2018.
- Peng, B., Quesnelle, J., Fan, H., and Shippole, E. Yarn: Efficient context window extension of large language models. In *International Conference on Learning Representations*, 2024.
- Pritchett, B. L. *Garden path phenomena and the grammatical basis of language processing*. Harvard University, 1987.
- Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., Sutskever, I., et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.
- Sakaguchi, K., Bras, R. L., Bhagavatula, C., and Choi, Y. Winogrande: An adversarial winograd schema challenge at scale. *Communications of the ACM*, 64(9):99–106, 2021.
- Shazeer, N. Glu variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.
- Springer, J. M., Kotha, S., Fried, D., Neubig, G., and Raghunathan, A. Repetition improves language model embeddings. In *International Conference on Learning Representations*, 2025.
- Su, J., Ahmed, M., Lu, Y., Pan, S., Bo, W., and Liu, Y. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.
- Tillet, P., Kung, H.-T., and Cox, D. Triton: an intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, pp. 10–19, 2019.
- Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- Xiong, W., Liu, J., Molybog, I., Zhang, H., Bhargava, P., Hou, R., Martin, L., Rungta, R., Sankararaman, K. A., Oguz, B., et al. Effective long-context scaling of foundation models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pp. 4643–4663, 2024.
- Xu, X., Tao, C., Shen, T., Xu, C., Xu, H., Long, G., Lou, J.-G., and Ma, S. Re-reading improves reasoning in large language models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 15549–15575, 2024.
- Yadav, V., Bethard, S., and Surdeanu, M. Quick and (not so) dirty: Unsupervised selection of justification sentences for multi-hop question answering. In *Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing (EMNLP-IJCNLP)*, pp. 2578–2589, 2019.
- Yang, S., Shen, Y., Wen, K., Tan, S., Mishra, M., Ren, L., Panda, R., and Kim, Y. PaTH attention: Position encoding via accumulating householder transformations. In *Annual Conference on Neural Information Processing Systems*, 2025.
- Ye, T., Dong, L., Xia, Y., Sun, Y., Zhu, Y., Huang, G., and Wei, F. Differential transformer. In *International Conference on Learning Representations*, 2025.
- Zellers, R., Holtzman, A., Bisk, Y., Farhadi, A., and Choi, Y. Hellaswag: Can a machine really finish your sentence? In *Proceedings of the 57th annual meeting of the association for computational linguistics*, pp. 4791–4800, 2019.
- Zhang, B. and Sennrich, R. Root mean square layer normalization. *Advances in neural information processing systems*, 32, 2019.
- Zhang, S., Zhao, Y., Geng, L., Cohan, A., Tuan, L. A., and Zhao, C. Diffusion vs. autoregressive language models: A text embedding perspective. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 4273–4303, 2025.

A. Further Related Work

Encoder-only Next Token Prediction (ENTP) (Ewer et al., 2024) performs next-token prediction with encoder-only Transformers, where the keys are naturally re-computed at each position. It demonstrates stronger sample efficiency on small-scale language modeling and in-context learning benchmarks. However, the per-token compute in ENTP scales quadratically with sequence length and cubically over full sequences, which presents challenges for scaling.

The following works are largely orthogonal to CASTLE. However, parts of their mathematical formulations resonate with CASTLE. We include them below for completeness.

PaTH Attention (Yang et al., 2025) is a novel positional encoding scheme which introduces data-dependent encodings based on cumulative products of Householder-like transformations. Each transformation is conditioned on the input, enabling the model to dynamically adapt positional information as the sequence progresses. PaTH is related to CASTLE in the sense that in inference formulation of PaTH, keys are also updated via a rank-1 update. However, both the update mechanism of keys and the parallel training formulation in PaTH differ substantially from those in CASTLE, and PaTH remains fundamentally a positional encoding, making it orthogonal to our approach. In addition, it is possible to combine CASTLE with PaTH to further improve performance, which could be a meaningful future direction.

Selective Attention (Leviathan et al., 2025) introduces a parameter-free modification where tokens can mask out irrelevant or outdated tokens, which could be potentially related to our application of the SiLU function in (6). By subtracting accumulated selection scores from the attention logits, selective attention reduces reliance on unneeded context. As a result, it achieves substantial memory and compute savings without degrading perplexity.

B. Experimental Details and Additional Results

B.1. Experimental Setup

We give details of our experimental setup in this section.

B.1.1. MODEL ARCHITECTURE

We use the improved Transformer architecture with SwiGLU (Shazeer, 2020), Rotary Positional Embeddings (Su et al., 2024), and RMSNorm (Zhang & Sennrich, 2019) following LLaMA (Touvron et al., 2023). More specifically, in each layer l , given the contextualized representations $\mathbf{X}^{(l)} \in \mathbb{R}^{L \times d_{\text{model}}}$ where L is the sequence length and d_{model} is the hidden dimension, then, $\mathbf{X}^{(l+1)}$ is obtained by

$$\begin{aligned} \mathbf{Y}^{(l)} &= \text{MultiHead-Attention}(\text{RMSNorm}(\mathbf{X}^{(l)})) \\ \mathbf{X}^{(l+1)} &= \text{SwiGLU}(\text{RMSNorm}(\mathbf{Y}^{(l)})), \end{aligned}$$

where the $\text{SwiGLU}(\mathbf{X}) = (\text{Swish}(\mathbf{X} \mathbf{W}_1) \odot (\mathbf{X} \mathbf{W}_2)) \mathbf{W}_3$. Here, $\mathbf{W}_1 \in \mathbb{R}^{d_{\text{model}} \times \frac{8}{3}d_{\text{model}}}$, $\mathbf{W}_2 \in \mathbb{R}^{d_{\text{model}} \times \frac{8}{3}d_{\text{model}}}$, $\mathbf{W}_3 \in \mathbb{R}^{\frac{8}{3}d_{\text{model}} \times d_{\text{model}}}$ are learnable parameters.

The function MultiHead-Attention is instantiated with either the standard multi-head causal attention or the multi-head CASTLE, or the multi-head CASTLE-SWL.

B.1.2. MODEL CONFIGURATION AND TRAINING RECIPE

We train models on four scales from scratch: small (0.16B), medium (0.35B), large (0.75B) and XL (1.3B), where the medium, large and XL baseline models follow the configurations of GPT-3 (Brown et al., 2020). For the small setting, we increase the number of heads and the hidden dimension in the original GPT-3 configuration to better align parameter counts between standard causal attention and CASTLE. The configurations of the models can be found in Table 4. To ensure a fair comparison between CASTLE and standard causal attention, we align the number of model parameters by adjusting the number of attention heads and keeping hidden dimensions and head dimensions invariant. This avoids changes to the representational upper bound of the models' hidden states and the behavior of RoPE, both of which depend on the hidden dimension. As shown in Appendix D, the number of parameters in CASTLE matches that of standard causal attention when the number of heads satisfies the relation $n_{\text{CASTLE}} = \frac{4}{7}n_{\text{standard}}$, where n_{CASTLE} and n_{standard} denote the number of heads in CASTLE and standard causal attention, respectively. For the small model, the baseline uses 14 heads. By setting CASTLE to 8 heads, we align the parameter counts. For the other settings, the baseline models use 16 heads. As $16 * \frac{4}{7} \approx 9.14$, we

choose 9 heads for CASTLE, resulting in a slightly smaller number of parameters than the baseline.

The model configurations and training recipes of CASTLE-SWL are identical to those of CASTLE.

The sliding window size for lookahead keys in CASTLE-SWL is set to 128 in CASTLE-SWL-S, and 512 in CASTLE-SWL-M, CASTLE-SWL-L, and CASTLE-SWL-XL. Ablation studies on sliding window sizes of CASTLE-SWL are provided in Appendix B.4.4.

Table 4. Configurations and learning hyper-parameters (batch size in tokens and learning rate) of the models which we trained. CASTLE-SWL has exactly the same configurations and training hyper-parameters with CASTLE on each model scale, and is omitted from this table for clarity.

Model Name	n_{params}	n_{layers}	d_{model}	n_{heads}	d	Batch Size	Learning Rate
Baseline-S	160M	12	896 (=14 * 64)	14	64	0.5M	6.0×10^{-4}
CASTLE-S	160M	12	896	8	64		
Baseline-M	353M	24	1024 (=16 * 64)	16	64	0.5M	3.0×10^{-4}
CASTLE-M	351M	24	1024	9	64		
Baseline-L	756M	24	1536 (=16 * 96)	16	96	0.5M	2.5×10^{-4}
CASTLE-L	753M	24	1536	9	96		
Baseline-XL	1.310B	24	2048 (=16 * 128)	16	128	0.5M	2.0×10^{-4}
CASTLE-XL	1.304B	24	2048	9	128		

We adopt the training hyper-parameters of (Dao & Gu, 2024). We use the AdamW optimizer (Loshchilov & Hutter, 2019) with $\beta_1 = 0.9$, $\beta_2 = 0.95$, weight decay rate coefficient 0.1, and gradient clipping coefficient 1.0. All models are trained with sequence length 2K and batch size 0.5M tokens. The small, medium, large and XL models use peak learning rates of 6.0×10^{-4} , 3.0×10^{-4} , 2.5×10^{-4} and 2.0×10^{-4} , respectively. All models are trained with 2000 warmup steps, and then, the cosine scheduler decays the learning rate to 10% of the peak learning rate. All models are trained on the FineWeb-Edu dataset (Lozhkov et al.) for 50 billion tokens. The efficient parallel training algorithm of CASTLE and CASTLE-SWL (forward pass in Algorithm 1 and backward pass in Algorithm 3) is implemented in Triton (Tillet et al., 2019).

B.2. Additional Downstream Evaluation Results

Evaluation results (0-shot) are provided in Table 5. 5-shot results have been shown in Table 2.

Table 5. Evaluation results (0-shot) for downstream tasks of different model scales. Each model is pretrained on FineWeb-Edu for 50B tokens. All values denote accuracy in percentage (%). The higher accuracy values are shown in bold. Hella.=HellaSwag, Wino.=Winogrande.

Model Name	ARC-C	ARC-E	BoolQ	Hella.	OBQA	PIQA	SciQA	Wino.	Avg.
Baseline-S	26.71	<u>54.76</u>	52.51	35.78	30.40	63.98	<u>81.00</u>	52.57	49.71
CASTLE-S	26.19	56.69	59.85	36.28	<u>31.60</u>	<u>64.25</u>	80.60	<u>52.25</u>	50.96
CASTLE-SWL-S	<u>26.62</u>	54.12	<u>59.60</u>	<u>35.97</u>	32.60	64.31	81.40	50.51	<u>50.64</u>
Baseline-M	28.58	60.90	53.61	43.01	33.40	67.95	85.30	50.91	52.96
CASTLE-M	30.20	<u>61.36</u>	58.01	<u>43.24</u>	34.60	<u>67.95</u>	85.70	<u>52.64</u>	54.21
CASTLE-SWL-M	<u>29.52</u>	61.41	<u>57.03</u>	43.76	33.00	68.12	85.70	53.83	<u>54.05</u>
Baseline-L	<u>32.59</u>	<u>65.07</u>	57.49	47.45	32.60	<u>70.51</u>	86.40	50.75	55.36
CASTLE-L	32.34	65.15	<u>57.65</u>	<u>47.87</u>	<u>35.60</u>	70.78	87.70	<u>53.51</u>	<u>56.32</u>
CASTLE-SWL-L	32.76	63.51	60.95	48.50	36.00	70.02	<u>87.60</u>	54.85	56.77
Baseline-XL	33.79	66.62	<u>61.04</u>	51.40	36.20	72.58	88.30	54.06	58.00
CASTLE-XL	<u>35.32</u>	<u>67.51</u>	62.81	52.15	<u>37.00</u>	70.67	<u>88.60</u>	56.59	<u>58.83</u>
CASTLE-SWL-XL	36.43	69.07	60.24	<u>51.99</u>	37.40	<u>71.27</u>	89.30	<u>55.09</u>	58.85

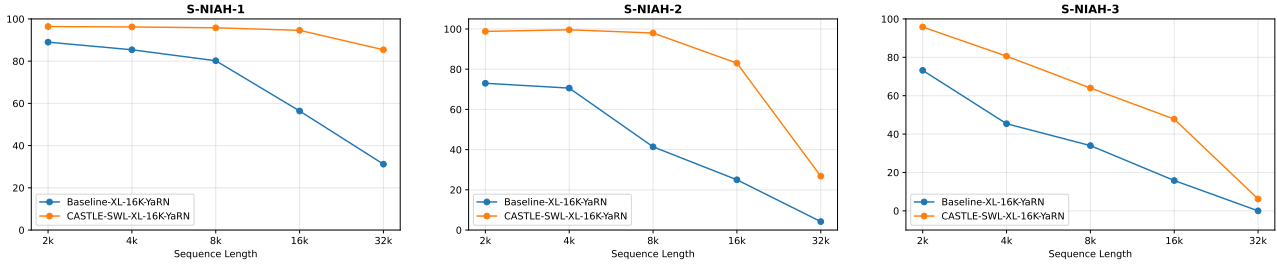


Figure 7. Results on the RULER S-NIAH tasks across different sequence lengths after applying YaRN.

B.3. Additional Results on Long-Context Tasks

B.3.1. TRAINING DETAILS

To test model’s capabilities on long-context tasks, we train XL size models on LongCrawl64 dataset (Buckman) for 50B tokens with sequence length 16K, peak learning rate 1.0×10^{-4} and batch size 2M tokens. RoPE base is set to 500K following (Xiong et al., 2024; Lin et al., 2025). Other optimizer hyper-parameters are the same as in Section B.1.2.

B.3.2. LENGTH EXTRAPOLATION

We evaluate the models’ compatibility with YaRN (Peng et al., 2024) for length extrapolation. Table 6 reports the performance of the models on the RULER S-NIAH tasks (Hsieh et al.) after applying YaRN to CASTLE-SWL-XL-16K and Baseline-XL-16K, both with yarn factor 2.0. Across all tasks, CASTLE-SWL-XL achieves substantially higher accuracy than Baseline-XL, indicating that CASTLE-SWL has markedly better compatibility with YaRN than the baseline MHA model.

Table 6. Accuracy on the RULER S-NIAH tasks across different sequence lengths when applying YaRN to CASTLE-SWL-XL-16K and Baseline-XL-16K. Significant higher accuracy is in bold.

	S-NIAH-1				
	2K	4K	8K	16K	32K
Baseline-XL-16K	89.0	85.4	80.2	56.4	31.2
CASTLE-SWL-XL-16K	96.4	96.2	95.8	94.6	85.4
	S-NIAH-2				
	2K	4K	8K	16K	32K
Baseline-XL-16K	73.0	70.6	41.4	25.0	4.2
CASTLE-SWL-XL-16K	98.8	99.6	98.0	83.0	26.8
	S-NIAH-3				
	2K	4K	8K	16K	32K
Baseline-XL-16K	73.2	45.4	34.0	15.8	0.0
CASTLE-SWL-XL-16K	95.8	80.6	64.0	47.8	6.2

B.4. Ablation Studies

We conduct ablation studies to better understand the contributions of different design components in CASTLE and CASTLE-SWL. These studies address the following key questions:

- Are causal keys necessary, or could lookahead keys alone suffice?
- Do the observed improvements arise from the mechanism of lookahead keys, or simply from increasing the number of keys?
- What is the role of the SiLU function in (6)?

- How do sliding window sizes of lookahead keys in CASTLE-SWL affect the performance?
- How do we design the mask M_t^U in the definition of CASTLE-SWL?
- Do CASTLE and CASTLE-SWL consistently outperform the baseline models across different token budgets?

We systematically investigate each question in the following sections.

B.4.1. ABLATIONS ON CAUSAL KEYS

As described in Section 1.3, CASTLE adopts a hybrid design that partitions keys into two groups: causal keys and lookahead keys. If all lookahead keys are replaced with causal keys, CASTLE reduces to standard causal attention. Thus, the performance gains demonstrated in Section 2 can be attributed to the introduction of lookahead keys. A natural follow-up question is whether causal keys are necessary, or if lookahead keys alone suffice.

To investigate this, we construct a variant of CASTLE in which all causal keys are removed. To ensure a fair comparison, we adjust the configurations so that the total parameter count remains the same, as shown in Table 7.

Table 7. Configurations of CASTLE and its variant without causal keys used in ablation study on causal keys.

CASTLE TYPE	n_{params}	n_{layers}	d_{model}	n_{heads}	d
CASTLE	120M	12	768	6	64
CASTLE w/o causal keys	120M	12	768	7	64

The above 2 models are both trained on FineWeb-Edu for 25B tokens for efficiency, using the same learning hyper-parameters with the small models in Section B.1.2. Their training and validation loss and perplexity are presented in Table 8.

Table 8. Training and validation loss and perplexity of CASTLE and its variant without causal keys. Each model is trained on FineWeb-Edu for 25B tokens.

	Train		Eval	
	Loss	PPL	Loss	PPL
CASTLE	2.913	18.417	2.920	18.541
CASTLE w/o causal keys	3.006	20.213	3.021	20.505

As shown in Table 8, removing causal keys results in a clear degradation in performance. This demonstrates that causal keys are indispensable in CASTLE.

While these results establish the necessity of both causal and lookahead keys, our current formulation in (6) employs a one-to-one pairing of a causal key and a lookahead key. An alternative design could involve grouping multiple causal keys with a single lookahead key, or vice versa. Exploring the optimal ratio between causal keys and lookahead keys is left for future work.

B.4.2. ABLATIONS ON THE NUMBER OF KEYS

As discussed in Section D, when CASTLE uses half as many heads as standard causal attention, its parameter count becomes $\frac{7}{8}$ of the baseline. To maintain comparable parameter counts, we adjust the number of heads accordingly. However, unlike the baseline where each head corresponds to one key, each head in CASTLE introduces two keys—one causal key and one lookahead key.

This design results in CASTLE models having 16, 18, 18, and 18 keys for the small, medium, large, and XL scales, respectively, compared to the corresponding baselines with 14, 16, 16, and 16 keys (Table 4). Thus, CASTLE naturally uses slightly more keys than its baseline counterparts. A natural question arises: are the observed improvements due to the introduction of lookahead keys, or simply from having more keys overall?

To disentangle this effect, we construct CASTLE variants with only half as many heads as their baselines, ensuring that the total number of keys ($n_{\text{CausalKeys}} + n_{\text{LookaheadKeys}}$) matches the baselines. This adjustment results in CASTLE having a notably smaller parameter count than the baselines.

We also did the same ablation studies for CASTLE-SWL.

For efficiency, we train the medium and XL variants on FineWeb-Edu for 25B tokens, using the same hyper-parameters as in Section B.1.2. Results are reported in Table 10 (CASTLE) and Table 11 (CASTLE-SWL).

As shown in Table 10, despite having clearly fewer parameters, both CASTLE-M-16 and CASTLE-XL-16 outperform their baselines: CASTLE-M-16 lags behind CASTLE-M by only 0.005 in validation loss, yet surpasses the baseline by 0.026; CASTLE-XL-16 trails CASTLE-XL by 0.008 in validation loss, while exceeding the baseline by 0.032. Similar performance of CASTLE-SWL can be observed in Table 11.

These findings confirm that CASTLE and CASTLE-SWL’s advantage stems from its mechanism of incorporating lookahead keys, rather than from increasing the number of keys from 16 to 18. This further consolidates the advantage of CASTLE and CASTLE-SWL.

Table 9. Configurations of baseline models, CASTLE, and its variants used in the ablation study on the number of keys. Baseline-M, Baseline-XL, CASTLE-M, and CASTLE-XL follow the same configurations as in Table 4. CASTLE-M-16 and CASTLE-XL-16 are constructed by reducing the number of heads in CASTLE-M and CASTLE-XL, respectively, so that the total number of keys ($n_{\text{LookaheadKeys}} + n_{\text{CausalKeys}}$) matches the number of keys of the corresponding baseline models. CASTLE-SWL-M, CASTLE-SWL-XL, CASTLE-SWL-M-16, CASTLE-SWL-XL-16 have identical configurations with CASTLE-M, CASTLE-XL, CASTLE-M-16, CASTLE-XL-16, respectively and are omitted from this table for clarity.

Model Name	n_{params}	n_{layers}	d_{model}	n_{heads}	$n_{\text{LookaheadKeys}} + n_{\text{CausalKeys}}$	d
Baseline-M	353M	24	1024 (=16 * 64)	16	16	64
CASTLE-M	351M	24	1024	9	18	64
CASTLE-M-16	340M	24	1024	8	16	64
Baseline-XL	1.310B	24	2048 (=16 * 128)	16	16	128
CASTLE-XL	1.304B	24	2048	9	18	128
CASTLE-XL-16	1.260B	24	2048	8	16	128

Table 10. Training and validation loss and perplexity of baseline models, CASTLE and CASTLE variants with the same number of keys as the baselines, after training for 25B tokens on FineWeb-Edu. The lowest loss and perplexity are shown in bold, and the second-lowest values are underlined.

	n_{params}	Train		Eval	
		Loss	PPL	Loss	PPL
Baseline-M	353M	2.740	15.483	2.742	15.523
CASTLE-M	351M	2.709	15.018	2.711	15.039
CASTLE-M-16	340M	<u>2.714</u>	<u>15.093</u>	<u>2.716</u>	<u>15.126</u>
Baseline-XL	1.310B	2.548	12.779	2.543	12.723
CASTLE-XL	1.304B	2.507	12.267	2.503	12.219
CASTLE-XL-16	1.260B	<u>2.514</u>	<u>12.349</u>	<u>2.511</u>	<u>12.316</u>

Table 11. Training and validation loss and perplexity of baseline models, CASTLE-SWL and CASTLE-SWL variants with the same number of keys as the baselines, after training for 25B tokens on FineWeb-Edu. The lowest loss and perplexity are shown in bold, and the second-lowest values are underlined.

	n_{params}	Train		Eval	
		Loss	PPL	Loss	PPL
Baseline-M	353M	2.740	15.483	2.742	15.523
CASTLE-SWL-M	351M	2.710	15.036	2.713	15.068
CASTLE-SWL-M-16	340M	<u>2.716</u>	<u>15.117</u>	<u>2.718</u>	<u>15.150</u>
Baseline-XL	1.310B	2.548	12.779	2.543	12.723
CASTLE-SWL-XL	1.304B	2.506	12.255	2.503	12.217
CASTLE-SWL-XL-16	1.260B	<u>2.513</u>	<u>12.339</u>	<u>2.508</u>	<u>12.276</u>

B.4.3. ABLATIONS ON SiLU FUNCTION IN (6)

In this section, we introduce the design of (6). We begin with the most direct formulation, which combines the causal-key and lookahead-key attention scores as

$$\mathbf{p}_t = \text{softmax}(\mathbf{s}_t^C - \mathbf{s}_t^U). \quad (14)$$

Since $\mathbf{s}_t^U$ may take positive or negative values and, by its definition in (5) together with the Gaussian initialization used when training from scratch, the distributions of $\mathbf{s}_t^U$ and $-\mathbf{s}_t^U$ are symmetric, the variants $\mathbf{p}_t = \text{softmax}(\mathbf{s}_t^C - \mathbf{s}_t^U)$ and $\mathbf{p}_t = \text{softmax}(\mathbf{s}_t^C + \mathbf{s}_t^U)$ should have equivalent performance. For consistency with the strategies discussed later in this section, we write it as $\mathbf{p}_t = \text{softmax}(\mathbf{s}_t^C - \mathbf{s}_t^U)$ here.

When training CASTLE-XL and CASTLE-SWL-XL without SiLU, i.e., replace (6) by (14), we consistently observed the loss blows up and shows NaN then. In particular, CASTLE-SWL-XL blows up around step 24300, whereas CASTLE-XL blows up earlier, around step 1500. Lowering the learning rate mitigates this instability. For example, CASTLE-SWL-XL without SiLU can remain stable up to at least 25B tokens when trained with peak learning rates of 1×10^{-4} or 5×10^{-5} . However, such reductions in learning rate lead to noticeably worse performance, as shown in Table 12.

Table 12. Ablation study of the SiLU function in (6). Removing SiLU causes training instability. More specifically, training CASTLE-SWL-XL without SiLU in (6) under the same learning rate (2×10^{-4}) as in Table 4 will have the loss curve blow up at around 24300 training step. Reducing the learning rate alleviates this instability issue but results in significant performance degradation.

	Learning Rate	Train		Eval	
		Loss	PPL	Loss	PPL
CASTLE-SWL-XL	2×10^{-4}	2.506	12.255	2.503	12.217
CASTLE-SWL-XL w/o SiLU	1×10^{-4}	2.523	12.468	2.520	12.424
CASTLE-SWL-XL w/o SiLU	5×10^{-5}	2.571	13.084	2.571	13.075

To address the instability observed with (14), we explored the following design variants:

- **Attempt I:** scale down the lookahead-key attention scores $\mathbf{s}_t^U$, i.e., with a constant $C > 1$,

$$\mathbf{p}_t = \text{softmax}(\mathbf{s}_t^C - \mathbf{s}_t^U / C). \quad (15)$$

We remark that $\mathbf{s}_t^U$ can be either positive or negative, and by the definition of $\mathbf{s}_t^U$ in (5) and the fact that we initialized model weights from Gaussian distribution when we train the models from scratch, defining $\mathbf{p}_t = \text{softmax}(\mathbf{s}_t^C - \mathbf{s}_t^U / C)$ and $\mathbf{p}_t = \text{softmax}(\mathbf{s}_t^C + \mathbf{s}_t^U / C)$ should have equivalent performance.

- **Attempt II:** Clipping $\mathbf{s}_t^U$ on both sides. For differentiability, we use tanh instead of clipping directly, i.e.,

$$\mathbf{p}_t = \text{softmax}(\mathbf{s}_t^C - \tanh(\mathbf{s}_t^U)). \quad (16)$$

By the same reason with Attempt I, defining $\mathbf{p}_t = \text{softmax}(\mathbf{s}_t^C - \tanh(\mathbf{s}_t^U))$ and $\mathbf{p}_t = \text{softmax}(\mathbf{s}_t^C + \tanh(\mathbf{s}_t^U))$ should have equivalent performance.

- **Attempt III:** Clipping $\mathbf{s}_t^U$ on one side. For differentiability, we use SiLU instead of one-side clipping directly. We have tried the following 2 variants:

1. $\mathbf{p}_t = \text{softmax}(\mathbf{s}_t^C - \text{SiLU}(\mathbf{s}_t^U))$
2. $\mathbf{p}_t = \text{softmax}(\mathbf{s}_t^C + \text{SiLU}(\mathbf{s}_t^U))$.

We remark that similarly to discussions in Attempt I and II above, $\mathbf{p}_t = \text{softmax}(\mathbf{s}_t^C - \text{SiLU}(\mathbf{s}_t^U))$ and $\mathbf{p}_t = \text{softmax}(\mathbf{s}_t^C - \text{SiLU}(-\mathbf{s}_t^U))$ should be equivalent design. However, it is clear that $\mathbf{p}_t = \text{softmax}(\mathbf{s}_t^C - \text{SiLU}(\mathbf{s}_t^U))$ and $\mathbf{p}_t = \text{softmax}(\mathbf{s}_t^C + \text{SiLU}(\mathbf{s}_t^U))$ are not equivalent because $\text{SiLU}(x)$ is not symmetric and $\text{SiLU}(-x) \neq -\text{SiLU}(x)$.

Among the design choices mentioned above, $\mathbf{p}_t = \text{softmax}(\mathbf{s}_t^C + \text{SiLU}(\mathbf{s}_t^U))$ shows even earlier blowing up than (14). More specifically, on the XL size model, its loss curve starts to increase at the 1100-th step. The other choices show stable training. We attach our test results of Attempt I and Attempt II below. Attempt III is what we did in CASTLE(-SWL).

Attempt I to solve instability of (14)

Our Attempt I primarily targets the instability that arises when training lookahead keys with a sliding window while using (14) in place of (6). To mitigate this issue, we scale s_t^U by a constant factor C , motivated by the intuition that excessively large s_t^U values may induce training instability under (14). By definition, the magnitude of s_t^U is tied to the lookahead-key window size, suggesting that C should naturally be chosen in relation to this window size. For a fair comparison, we adopt the same window size for both Attempt II and CASTLE-SWL-XL. In our experiments for Attempt II, we set $C = \sqrt{W}$, where W is the window size of lookahead keys.

The performance of Attempt I (which incorporates the modification in (15)) is reported in Tables 13 and 14. In terms of training and validation loss, Attempt I improves slightly over the baseline but remains significantly worse than CASTLE-SWL-XL. Downstream evaluation results follow the same pattern: Attempt I outperforms the baseline on average, yet consistently underperforms relative to CASTLE-SWL-XL. The fact that Attempt I surpasses the baseline demonstrates the effectiveness of the lookahead mechanism. However, its performance deficit compared to CASTLE-SWL-XL may stem from the scaling factor which may constrain the model’s expressivity.

We note that in Attempt I both the lookahead-key window size and the scaling factor C can influence model performance. A more systematic study, including making C a learnable parameter, is left for future work.

Table 13. Training and validation loss and perplexity comparison between Attempt I to solve instability of (14) (using (15)) and the baseline model. Each model is trained on FineWeb-Edu for 50B tokens.

	n_{params}	Train		Eval	
		Loss	PPL	Loss	PPL
Baseline-XL	1.310B	2.430	11.360	2.426	11.309
CASTLE-SWL-XL	1.304B	2.401	11.036	2.389	10.900
Attempt I (15)	1.304B	<u>2.424</u>	<u>11.286</u>	<u>2.417</u>	<u>11.215</u>

Table 14. Downstream evaluation (5-shot) comparison between Attempt I to solve instability of (14) (using (15)) and the baseline model. Each model is trained on FineWeb-Edu for 50B tokens.

Model Name	ARC-C	ARC-E	BoolQ	Hella.	OBQA	PIQA	Wino.	Avg.
Baseline-XL	35.58	65.78	61.07	<u>50.84</u>	<u>36.20</u>	<u>71.27</u>	52.72	53.35
CASTLE-SWL-XL	38.99	70.08	61.74	52.35	37.20	72.52	56.75	55.66
Attempt I (15)	<u>37.29</u>	<u>68.64</u>	<u>61.71</u>	50.26	35.20	69.97	<u>56.20</u>	<u>54.18</u>

Attempt II to solve instability of (14)

Table 15 reports the training and validation losses and perplexities for Attempt II (which incorporates the modification in (16)) compared with the baseline model. As shown, Attempt II achieves slightly lower training and validation losses than the baseline, but remains noticeably worse than CASTLE-M. Downstream evaluation results in Table 16 also exhibit a similar trend, i.e., Attempt II outperforms Baseline-M on average, yet still lags behind CASTLE-M across most tasks.

Following a similar trend with Attempt I, the fact that Attempt II outperforms the baseline demonstrates the effectiveness of the lookahead mechanism again. However, its lower performance compared to CASTLE-M may come from the clipping on both sides (using tanh) which overly constrains the model’s expressivity.

Table 15. Training and validation loss and perplexity comparison between Attempt II to solve instability of (14) (using (16)) and the baseline model. Each model is trained on FineWeb-Edu for 50B tokens.

	n_{params}	Train		Eval	
		Loss	PPL	Loss	PPL
Baseline-M	353M	2.641	14.030	2.639	14.004
CASTLE-M	351M	2.616	13.684	2.615	13.665
Attempt II (16)	351M	2.636	13.961	2.634	13.926

Table 16. Downstream evaluation (5-shot) comparison between Attempt II to solve instability of (14) (using (16)) and the baseline model. Each model is trained on FineWeb-Edu for 50B tokens.

Model Name	ARC-C	ARC-E	BoolQ	Hella.	OBQA	PIQA	Wino.	Avg.
Baseline-M	31.06	62.46	48.47	42.83	<u>33.00</u>	68.39	51.78	48.28
CASTLE-M	32.17	64.06	<u>54.62</u>	43.47	33.80	69.48	52.49	50.01
Attempt II (16)	<u>31.83</u>	<u>63.47</u>	59.94	42.31	32.00	<u>68.55</u>	50.91	<u>49.86</u>

B.4.4. ABLATIONS ON SLIDING WINDOW SIZE IN LOOKAHEAD KEYS OF CASTLE-SWL

We investigate the effect of sliding window size on CASTLE-SWL across different model scales. For efficiency, each model is trained on the FineWeb-Edu dataset for 25B tokens. We train CASTLE-SWL with window sizes of 64, 128, 256, 512, and 1024, and report both training and validation loss and perplexity. The results for small, medium, large, and XL models are shown in Table 17, Table 18, Table 19, and Table 20, respectively. Throughout, we use the number following “SWL” to denote the sliding window size, e.g., CASTLE-SWL128-S refers to CASTLE-SWL-S with a window size of 128.

As shown in Table 17, a window size of 128 achieves the best performance for the small model. For medium, large, and XL models (Tables 18, 19, and 20), the optimal performance is obtained with a window size of 512. Based on these findings, we adopt window sizes of 128, 512, 512, and 512 for small, medium, large, and XL models, respectively, in the experiments presented in Section 2.

Overall, CASTLE-SWL performance is not sensitive to the choice of sliding window size in the range [128, 1024]. For example, for the medium model, the best sliding window size (512) reduces validation loss by 0.0297 compared to the baseline, but the gap between the best (512) and worst window sizes (256) is only 0.0038. Similarly, for the XL model, the best sliding window (512) improves upon the baseline by 0.0406 while the difference between the best (512) and worst (128) sliding window sizes is just 0.0099. These results suggest that while CASTLE-SWL consistently improves over the baselines across all model scales and sliding window sizes tested, and its performance is relatively robust to the choices of window sizes.

However, for the medium, large, and XL models, using a window size of 64 results in substantial performance degradation. In both training and validation metrics, CASTLE-SWL64 performs at least 0.01 worse than its counterpart CASTLE-SWL512. Moreover, in downstream evaluation, CASTLE-SWL64 shows a clear disadvantage not only compared to CASTLE-SWL512, but even relative to the baseline model. The downstream evaluation results are presented in Table 21.

Table 17. Ablations on sliding window sizes for CASTLE-SWL-S. Training and validation loss and perplexity of baseline models, CASTLE-SWL-S with different sliding window sizes after training for 25B tokens on FineWeb-Edu are reported. The lowest loss and perplexity are shown in bold, and the second-lowest values are underlined.

	n_{params}	Train		Eval	
		Loss	PPL	Loss	PPL
Baseline-S	160M	2.892	18.037	2.901	18.197
CASTLE-SWL64-S	160M	2.884	17.888	2.890	18.024
CASTLE-SWL128-S	160M	2.883	17.859	2.889	17.971
CASTLE-SWL256-S	160M	2.888	17.952	2.895	18.092
CASTLE-SWL512-S	160M	2.885	17.910	2.892	18.023
CASTLE-SWL1024-S	160M	2.885	17.901	2.892	18.031

Table 18. Ablations on sliding window sizes for CASTLE-SWL-M. Training and validation loss and perplexity of baseline models, CASTLE-SWL-M with different sliding window sizes after training for 25B tokens on FineWeb-Edu are reported. The lowest loss and perplexity are shown in bold, and the second-lowest values are underlined.

	n_{params}	Train		Eval	
		Loss	PPL	Loss	PPL
Baseline-M	353M	2.740	15.483	2.742	15.523
CASTLE-SWL64-M	351M	2.726	15.273	2.728	15.306
CASTLE-SWL128-M	351M	2.715	15.098	2.716	15.124
CASTLE-SWL256-M	351M	2.715	15.112	2.716	15.127
CASTLE-SWL512-M	351M	2.710	15.036	2.713	15.068
CASTLE-SWL1024-M	351M	<u>2.713</u>	<u>15.071</u>	<u>2.715</u>	<u>15.103</u>

Table 19. Ablations on sliding window sizes for CASTLE-SWL-L. Training and validation loss and perplexity of baseline models, CASTLE-SWL-L with different sliding window sizes after training for 25B tokens on FineWeb-Edu are reported. The lowest loss and perplexity are shown in bold, and the second-lowest values are underlined.

	n_{params}	Train		Eval	
		Loss	PPL	Loss	PPL
Baseline-L	756M	2.740	15.483	2.742	15.523
CASTLE-SWL64-L	753M	2.608	13.572	2.607	13.552
CASTLE-SWL128-L	753M	2.597	13.425	2.596	13.411
CASTLE-SWL256-L	753M	2.589	13.314	2.587	13.290
CASTLE-SWL512-L	753M	2.582	13.219	2.580	13.202
CASTLE-SWL1024-L	753M	<u>2.582</u>	<u>13.229</u>	<u>2.581</u>	<u>13.209</u>

Table 20. Ablations on sliding window sizes for CASTLE-SWL-XL. Training and validation loss and perplexity of baseline models, CASTLE-SWL-XL with different sliding window sizes after training for 25B tokens on FineWeb-Edu are reported. The lowest loss and perplexity are shown in bold, and the second-lowest values are underlined.

	n_{params}	Train		Eval	
		Loss	PPL	Loss	PPL
Baseline-XL	1.310B	2.548	12.779	2.543	12.723
CASTLE-SWL64-XL	1.304B	2.530	12.548	2.525	12.491
CASTLE-SWL128-XL	1.304B	2.517	12.393	2.513	12.339
CASTLE-SWL256-XL	1.304B	2.514	12.353	2.510	12.300
CASTLE-SWL512-XL	1.304B	2.506	12.255	2.503	12.217
CASTLE-SWL1024-XL	1.304B	<u>2.514</u>	<u>12.351</u>	<u>2.509</u>	<u>12.294</u>

Table 21. Downstream evaluation (5-shot) comparison among CASTLE-SWL64-XL, CASTLE-SWL512-XL and Baseline-XL. Each model is trained on FineWeb-Edu for 25B tokens.

Model Name	ARC-C	ARC-E	BoolQ	Hella.	OBQA	PIQA	Wino.	Avg.
Baseline-XL	<u>33.96</u>	64.06	61.99	<u>46.92</u>	33.40	<u>69.10</u>	<u>52.88</u>	51.76
CASTLE-SWL512-XL	34.81	69.32	55.35	49.50	36.60	71.00	53.99	52.94
CASTLE-SWL64-XL	33.11	<u>64.98</u>	<u>57.95</u>	44.51	32.60	69.04	52.09	50.61

B.4.5. ABLATIONS ON BASELINE MODELS WITH SLIDING WINDOW

In this section, we provide evidence that the performance gains of CASTLE-SWL over baseline models stem from the core mechanism of lookahead keys, rather than from the mere use of a sliding window.

To isolate the effect of sliding windows, we augment Baseline-S and Baseline-L with sliding-window sizes of 128 and 512, respectively, matching the window sizes used for lookahead keys in CASTLE-SWL-S and CASTLE-SWL-L. We refer to these modified models as Baseline-SW-S and Baseline-SW-L.

All models are trained on 50B tokens. Their training and validation loss and perplexity are reported in Table 22, and downstream evaluation results are presented in Table 23.

We observe that both Baseline-SW-S and Baseline-SW-L experience substantial degradation in training and validation loss. Their downstream task performance is either comparable to or worse than the original baselines.

These results indicate that applying a sliding window to lookahead keys is fundamentally different from applying it to the causal keys in standard causal attention. And the advantages of CASTLE-SWL cannot be attributed simply to the use of a sliding-window mechanism.

Table 22. Training and validation loss and perplexity for models with CASTLE, standard causal attention and standard causal attention with sliding windows. Each model is trained on FineWeb-Edu for 50B tokens. The best and second best results are highlighted in bold and underline, respectively.

	n_{params}	Train		Eval	
		Loss	PPL	Loss	PPL
Baseline-S	160M	2.795	16.364	2.798	16.411
Baseline-SW-S	160M	2.861	17.487	2.868	17.608
CASTLE-SWL-S	160M	2.786	16.213	2.790	16.273
Baseline-L	756M	2.513	12.346	2.507	12.269
Baseline-SW-L	756M	2.526	12.503	2.524	12.480
CASTLE-SWL-L	753M	2.476	11.890	2.471	11.832

Table 23. Evaluation results (5-shot) for downstream tasks of models with CASTLE, standard causal attention and standard causal attention with sliding windows. The higher accuracy values are shown in bold. All values denote accuracy in percentage (%). Each model is pretrained on FineWeb-Edu for 50B tokens. Hella.=HellaSwag, Wino.=Winogrande.

Model Name	ARC-C	ARC-E	BoolQ	Hella.	OBQA	PIQA	Wino.	Avg.
Baseline-S	25.68	54.97	56.09	33.81	28.20	63.98	52.57	45.04
Baseline-SW-S	26.28	53.91	54.62	35.51	29.00	62.73	53.20	45.04
CASTLE-SWL-S	27.39	56.19	56.09	35.46	30.20	64.85	51.22	45.91
Baseline-L	33.36	63.64	59.24	46.16	33.40	69.53	54.06	51.34
Baseline-SW-L	32.94	65.36	59.24	46.05	32.40	69.31	54.14	51.35
CASTLE-SWL-L	36.26	65.53	60.58	48.55	34.80	69.10	53.67	52.64

B.4.6. ABLATIONS ON MASK M_t^U IN THE DEFINITION OF CASTLE

In this section, we describe why we define M_t^U as it is in (3). We have tested all the following 3 cases of $M_t^U \in \mathbb{R}^{t \times t}$:

- Option I: removing M_t^U in (2), which is equivalent to defining

$$[M_t^U]_{ij} = 0 \quad \text{for any } i, j \in [t]. \quad (17)$$

When generating the $(t+1)$ -th token, Option I allows lookahead keys of each token s to have access to information from representations of all tokens in $\{1, \dots, t\}$.

- Option II: define M_t^U as the causal mask, i.e.,

$$[M_t^U]_{ij} = 0 \quad \text{if } i \geq j \quad \text{and} \quad [M_t^U]_{ij} = -\infty \quad \text{otherwise.} \quad (18)$$

In Option II, lookahead keys no longer look ahead, i.e., when generating the $(t+1)$ -th token, Option I restricts lookahead keys of each token s to have access to information only from tokens in $\{1, \dots, s\}$.

- Option III: define M_t^U as in (3). This is what we choose for CASTLE. More specifically, when generating the $(t+1)$ -th token, Option I allows lookahead keys of each token s to have access to information from representations of its subsequent tokens in $\{s+1, \dots, t\}$

It is shown that both Option I and Option II lead to worse training and validation losses compared with the baseline. Although their downstream evaluation performance is comparable to the baseline, it remains significantly below that of CASTLE.

A likely explanation for the poor performance of Option I is that the model learns features from preceding and subsequent tokens in different ways. Allowing lookahead keys to attend to both directions simultaneously may introduce conflicts and then degrade learning.

The poor performance of Option II highlights the importance of the lookahead mechanism itself, i.e., the improvement of lookahead keys does not come from the structure in (6) in which attention scores from different heads are combined together. Instead, the superiority of lookahead keys comes from the “lookahead” mechanism itself, i.e., allowing keys to have access to information after it (while remaining its autoregressive property).

Table 24. Training and validation loss and perplexity comparison among Option I (17) and Option II (18) of M_t^U , CASTLE and the baseline model. Each model is trained on FineWeb-Edu for 25B tokens.

	n_{params}	Train		Eval	
		Loss	PPL	Loss	PPL
Baseline-M	353M	<u>2.740</u>	<u>15.483</u>	<u>2.742</u>	<u>15.523</u>
CASTLE-M	351M	2.709	15.011	2.711	15.039
Option I (17)	351M	2.744	15.545	2.746	15.573
Option II (18)	351M	2.758	15.764	2.761	15.822

Table 25. Downstream evaluation (0-shot) comparison among Option I (17) and Option II (18) of M_t^U , CASTLE and the baseline model. Each model is trained on FineWeb-Edu for 25B tokens.

Model Name	ARC-C	ARC-E	BoolQ	Hella.	OBQA	PIQA	Wino.	Avg.
Baseline-M	27.90	59.09	58.13	<u>40.08</u>	<u>31.80</u>	66.05	51.07	44.72
CASTLE-M	28.33	59.13	59.05	40.79	31.60	66.54	50.51	45.02
Option I (17)	28.16	57.83	54.34	39.72	34.80	66.16	53.04	44.70
Option II (18)	28.58	59.89	<u>58.26</u>	39.90	29.40	<u>66.21</u>	<u>51.62</u>	<u>44.83</u>

B.4.7. EXPERIMENTS ON DIFFERENT TOKEN-BUDGETS

We trained Baseline-XL, CASTLE-XL and CASTLE-SWL on 5B, 10B, 20B, 30B and 50B tokens separately. The results are reported in Table 26 and Table 27 together with the scaling curves in Figure 8. It is shown that both CASTLE and CASTLE-SWL consistently outperform the Baseline-XL model across all training-token budgets from 5B to 50B tokens. The scaling trends appear approximately linear when plotted in log-log space. Overall, the token-scaling experiments validate that CASTLE and CASTLE-SWL achieves better scaling properties than the baseline, indicating superior sample efficiency.

Table 26. Training loss for XL-size models trained with different token budgets.

Model	5B	10B	20B	30B	50B
Baseline-XL	2.798	2.650	2.544	2.485	2.430
CASTLE-SWL-XL	2.733	2.599	<u>2.497</u>	2.449	2.401
CASTLE	<u>2.736</u>	<u>2.602</u>	2.491	<u>2.450</u>	2.401

Table 27. Validation loss for XL-size models trained with different token budgets.

Model	5B	10B	20B	30B	50B
Baseline-XL	2.804	2.650	2.540	2.480	2.426
CASTLE-SWL-XL	2.737	2.597	<u>2.497</u>	<u>2.444</u>	2.389
CASTLE	<u>2.739</u>	<u>2.602</u>	2.491	2.443	<u>2.391</u>

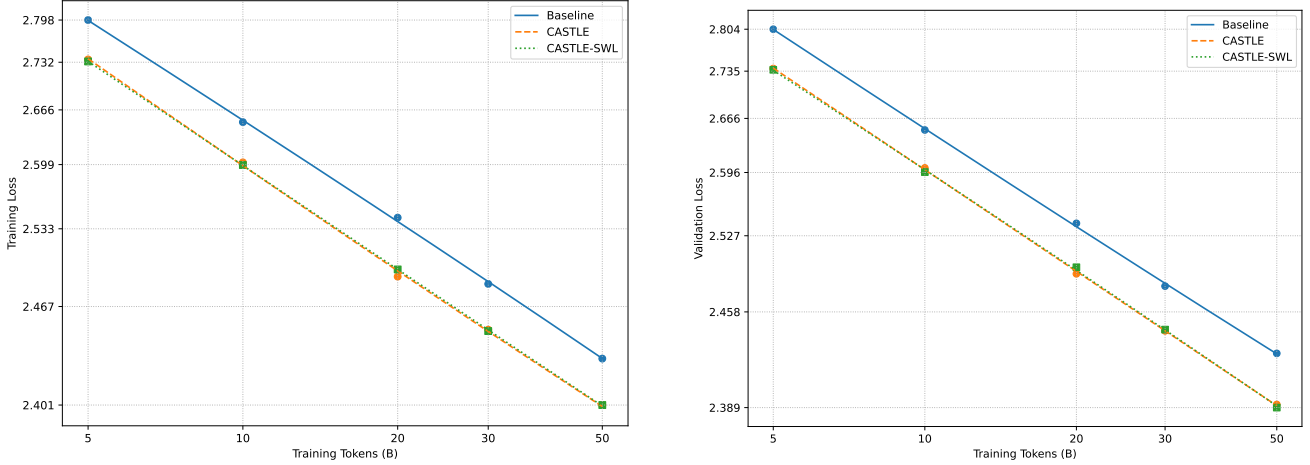


Figure 8. Scaling behavior of XL-size models as a function of training tokens. The x -axis is plotted in log-scale over the token budget, while the y -axis uses the transformation $\log(\text{loss} - 2.15)$. Both CASTLE and CASTLE-SWL exhibit consistently better scaling trends than the Baseline-XL model in training and validation loss.

B.5. Additional Loss Curves

This section presents additional training and validation loss curves for the small, medium, and large models, while the loss curves for the XL models are already shown in Figure 4. Each figure compares the baseline with CASTLE and CASTLE-SWL.

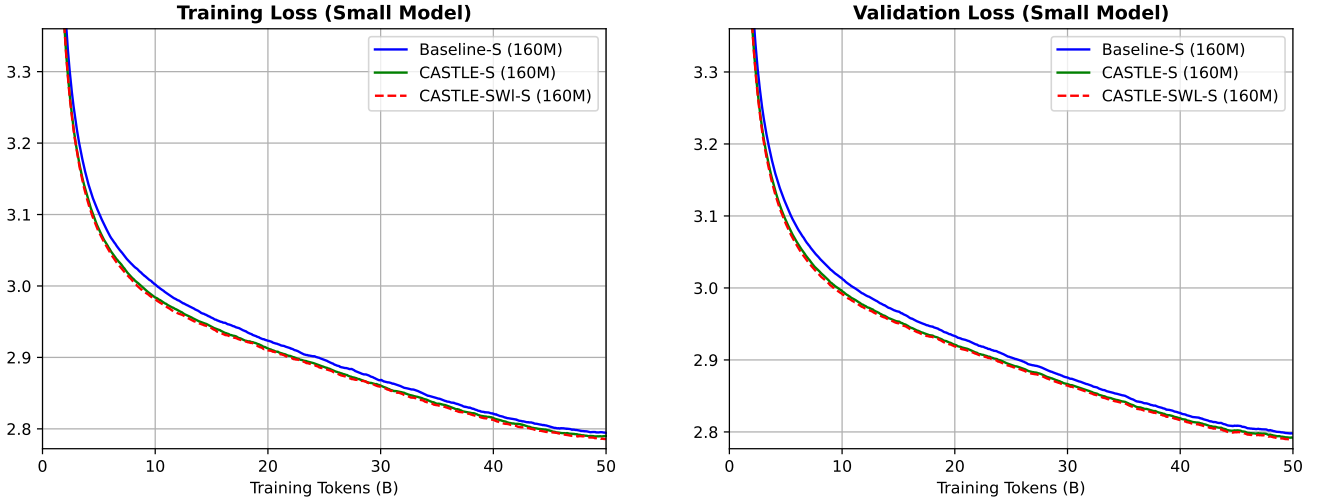


Figure 9. Training and validation loss curves of small models. Training loss curve is smoothened with a moving window of 2000 training steps. Validation loss is evaluated every 100 training steps on 40M tokens, and its curve is smoothened by a moving window of 20 evaluation intervals. As seen in Table 1 and in comparison with Figure 10, Figure 11 and Figure 4, CASTLE yields only marginal improvements over the baseline on small models. A likely explanation is that this is because the benefit of lookahead keys may lie in helping models capture global dependencies, but small models are capacity-limited and can primarily extract local features, making global relations less useful at this scale.

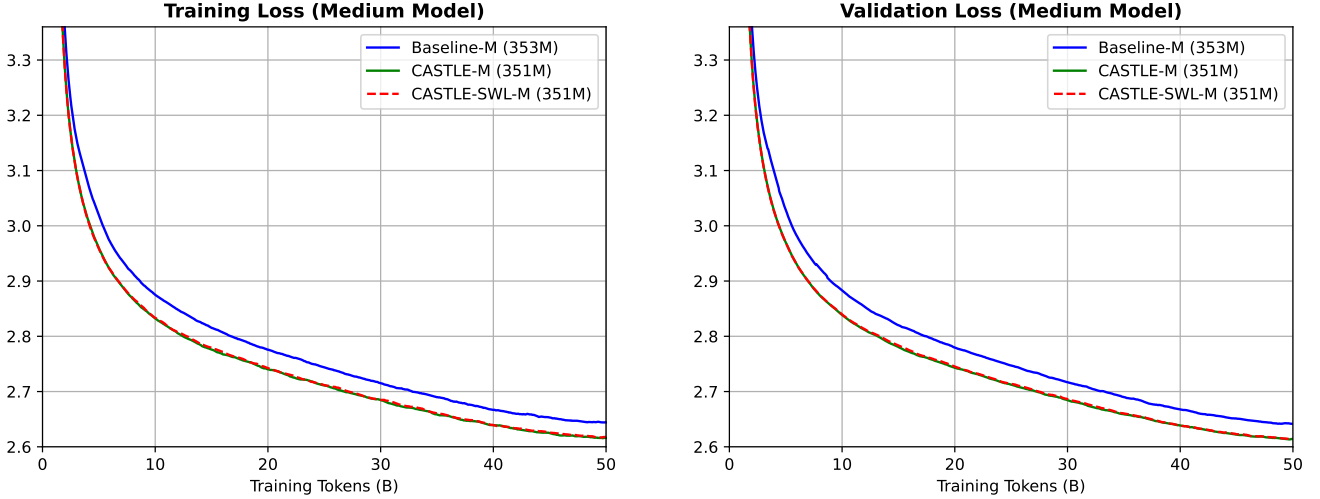


Figure 10. Training and validation loss curves of medium models. Training loss curve is smoothened with a moving window of 2000 training steps. Validation loss is evaluated every 100 training steps on 40M tokens, and its curve is smoothened by a moving window of 20 evaluation intervals. After 50B training tokens, CASTLE-M achieves a 0.0294 lower training loss and a 0.0245 lower validation loss compared to Baseline-M, while CASTLE-SWL-M achieves a 0.0232 lower training loss and a 0.0241 lower validation loss compared to Baseline-M

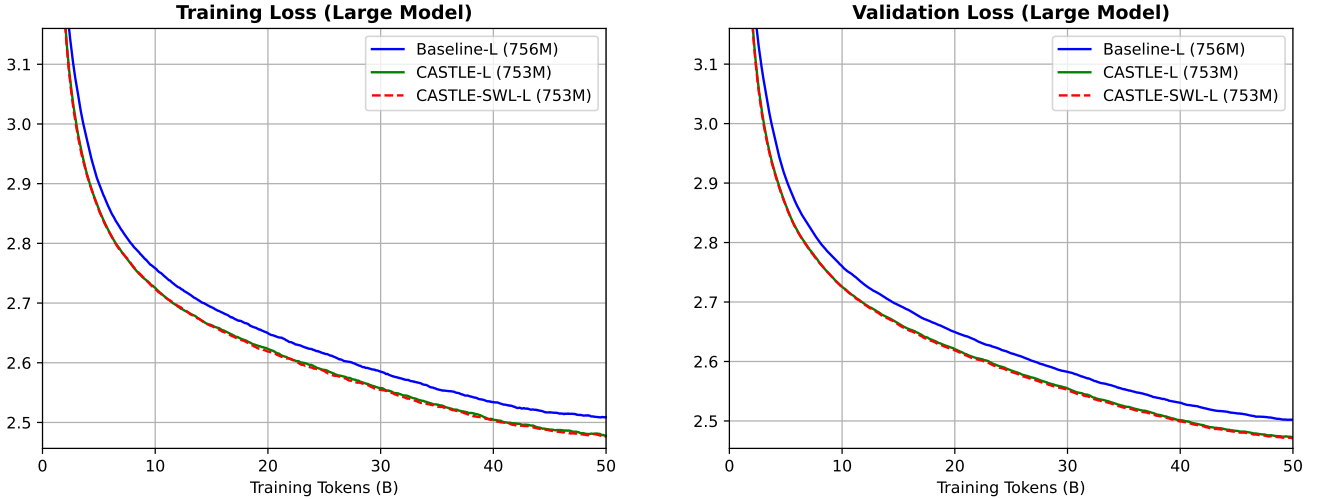


Figure 11. Training and validation loss curves of large models. Training loss curve is smoothened with a moving window of 2000 training steps. Validation loss is evaluated every 100 training steps on 40M tokens, and its curve is smoothened by a moving window of 20 evaluation intervals. After 50B training tokens, CASTLE-L achieves a 0.0371 lower training loss and a 0.0356 lower validation loss compared to Baseline-L, while CASTLE-SWL-L achieves a 0.0376 lower training loss and a 0.0366 lower validation loss compared to Baseline-L

B.6. Efficiency

• **Inference Efficiency:** We implement our CASTLE and CASTLE-SWL inference kernels in Triton (Tillet et al., 2019) and measure the execution time of a single CASTLE(-SWL) block which as in Table 4 has 9 heads, head dimension 128, hidden dimension 2048. For comparison, we report the runtime of the FlashAttention-2 Triton implementation of standard causal attention block used in the Baseline-XL model, which uses 16 heads with the same head and hidden dimensions. We use batch size 256. The experiments are conducted on a single H100 GPU. The results are measured in milliseconds and summarized in Table 28. It is shown that CASTLE-SWL consistently runs faster than CASTLE-XL across all sequence lengths. CASTLE-SWL narrows the gap to the FlashAttention-2 Baseline-XL implementation at longer sequence lengths.

We remark that our current inference kernels for CASTLE-SWL and CASTLE are not fully optimized. In particular, they exhibit lower memory–bandwidth utilization compared to the highly optimized FlashAttention-2 Triton kernels. This suggests that the inference performance of CASTLE(-SWL) still has substantial room for improvement.

Table 28. Inference efficiency (in milliseconds) comparison between FlashAttention-2 Triton implementation of standard causal attention and Triton implementation for inference kernel of CASTLE.

	Baseline-XL	CASTLE-SWL-XL	CASTLE-XL
2K	1.4026	1.7986	2.4718
4K	2.7691	3.3795	4.9139
8K	5.5111	6.4824	9.8132
16K	11.0117	12.5904	19.6271

•**Training Efficiency:** We compare training throughput (tokens/s) of the CASTLE-SWL-XL, CASTLE-XL and Baseline-XL models. The experiments are conducted on a single H100 GPU. Results are provided in Table 29. CASTLE-SWL achieves notably higher throughput than CASTLE-XL. At longer sequences, the difference between CASTLE-SWL-XL and Baseline-XL also narrows.

We remark that we made no attempts to optimize the training kernels. Developing more efficient kernel implementations is left for future work.

Table 29. Training throughput (tokens/s) comparison between FlashAttention-2 Triton implementation of standard causal attention and Triton implementation for training kernel of CASTLE.

SeqLen	BS	Baseline-XL	CASTLE-SWL-XL	CASTLE-XL
2K	8	39817	24651	17537
4K	4	32926	21339	10472
8K	2	24547	17338	5765
16K	1	16386	12933	3027

C. Proof of Theorem 1.1

First, recall the notations in Section 1.4. More specifically, consider inputs $\mathbf{X}^L = \begin{pmatrix} \mathbf{x}_1 \\ \mathbf{x}_2 \\ \vdots \\ \mathbf{x}_L \end{pmatrix} \in \mathbb{R}^{L \times d_{\text{model}}}$, where $\mathbf{x}_t$ is the representation of the t -th input token, L is the sequence length and d_{model} is the hidden dimension.

For each $1 \leq t \leq L$, denote

$$\begin{aligned} \mathbf{q}_t^U &= \mathbf{x}_t \mathbf{W}_Q^U, & \mathbf{k}_t^U &= \mathbf{x}_t \mathbf{W}_K^U, & \mathbf{v}_t^U &= \mathbf{x}_t \mathbf{W}_V^U, \\ \mathbf{q}_t^C &= \mathbf{x}_t \mathbf{W}_Q^C, & \mathbf{k}_t^C &= \mathbf{x}_t \mathbf{W}_K^C, & \mathbf{v}_t^C &= \mathbf{x}_t \mathbf{W}_V^C. \end{aligned}$$

$$\begin{aligned} \mathbf{Q}_t^U &= \begin{pmatrix} \mathbf{q}_1^U \\ \mathbf{q}_2^U \\ \vdots \\ \mathbf{q}_t^U \end{pmatrix} = \mathbf{X}^t \mathbf{W}_Q^U, & \mathbf{K}_t^U &= \begin{pmatrix} \mathbf{k}_1^U \\ \mathbf{k}_2^U \\ \vdots \\ \mathbf{k}_t^U \end{pmatrix} = \mathbf{X}^t \mathbf{W}_K^U, & \mathbf{V}_t^U &= \begin{pmatrix} \mathbf{v}_1^U \\ \mathbf{v}_2^U \\ \vdots \\ \mathbf{v}_t^U \end{pmatrix} = \mathbf{X}^t \mathbf{W}_V^U, \\ \mathbf{Q}_t^C &= \begin{pmatrix} \mathbf{q}_1^C \\ \mathbf{q}_2^C \\ \vdots \\ \mathbf{q}_t^C \end{pmatrix} = \mathbf{X}^t \mathbf{W}_Q^C, & \mathbf{K}_t^C &= \begin{pmatrix} \mathbf{k}_1^C \\ \mathbf{k}_2^C \\ \vdots \\ \mathbf{k}_t^C \end{pmatrix} = \mathbf{X}^t \mathbf{W}_K^C, & \mathbf{V}_t^C &= \begin{pmatrix} \mathbf{v}_1^C \\ \mathbf{v}_2^C \\ \vdots \\ \mathbf{v}_t^C \end{pmatrix} = \mathbf{X}^t \mathbf{W}_V^C. \end{aligned}$$

And M_t^C , $\widetilde{M}_t^C$ and M_t^U are t -by- t mask matrices. M_t^C is the t -by- t causal mask which prevents tokens from attending to their future tokens, i.e., $[M_t^C]_{ij} = 0$ if $i \geq j$ and $[M_t^C]_{ij} = -\infty$ otherwise; $[\widetilde{M}_t^C]_{ij} = 1$ if $i \geq j$ and $[\widetilde{M}_t^C]_{ij} = 0$ otherwise. For CASTLE, M_t^U is defined in (3) and for CASTLE-SWL, M_t^U is defined in (8).

For the projection matrices of the entire sequence X^L , we drop L as in Theorem 1.1 for simplicity, i.e.,

$$\begin{aligned} Q^U &= Q_L^U = X^L W_Q^U, & K^U &= K_L^U = X^L W_K^U, & V^U &= V_L^U = X^L W_V^U, \\ Q^C &= Q_L^C = X^L W_Q^C, & K^C &= K_L^C = X^L W_K^C, & V^C &= V_L^C = X^L W_V^C. \end{aligned}$$

And $M^U = M_L^U$, $M^C = M_L^C$. Then, M_t^U is a t -by- t submatrix of $M^U = M_L^U$. Similarly, M_t^C is also a submatrix of M^C .

Consider when we are generating the $(t+1)$ -th token. As in (2)

$$U^t = \begin{pmatrix} u_1^t \\ u_2^t \\ \vdots \\ u_t^t \end{pmatrix} = \sigma\left(\frac{Q_t^U K_t^{U\top}}{\sqrt{d}} + M_t^U\right) V_t^U \in \mathbb{R}^{t \times d}$$

Then, the lookahead-key attention scores as in (5) are

$$s_t^U = \frac{q_t^C U^{t\top}}{\sqrt{d}} = \frac{q_t^C V_t^{U\top} \left(\sigma\left(\frac{Q_t^U K_t^{U\top}}{\sqrt{d}} + M_t^U\right) \right)^\top}{\sqrt{d}}. \quad (19)$$

We will need the following lemma to proceed.

Lemma C.1. For any vector $\mathbf{a} \in \mathbb{R}^{1 \times t}$, let $\tilde{\mathbf{a}} = (\mathbf{a}, \mathbf{0}^{1 \times (L-t)}) \in \mathbb{R}^{1 \times L}$, where $\mathbf{0}^{1 \times (L-t)}$ is the all-zeros vector of size $(1, L-t)$. Then,

$$\left(\mathbf{a} \left(\sigma\left(\frac{Q_t^U K_t^{U\top}}{\sqrt{d}} + M_t^U\right) \right)^\top, \mathbf{0}^{1 \times (L-t)} \right) = \tilde{\mathbf{a}} \left(\sigma\left(\frac{Q^U K^{U\top}}{\sqrt{d}} + M^U\right) \right)^\top.$$

Proof of Lemma C.1. The proof is straightforward by the fact that

1. The upper triangular entries of the transposed matrix $\left(\sigma\left(\frac{Q^U K^{U\top}}{\sqrt{d}} + M^U\right) \right)^\top$ are all 0 by the definition of M^U .
2. The matrix $\sigma\left(\frac{Q_t^U K_t^{U\top}}{\sqrt{d}} + M_t^U\right)$ equals an upper-left block of the matrix $\sigma\left(\frac{Q^U K^{U\top}}{\sqrt{d}} + M^U\right)$, i.e.,

$$\sigma\left(\frac{Q_t^U K_t^{U\top}}{\sqrt{d}} + M_t^U\right) = \left[\sigma\left(\frac{Q^U K^{U\top}}{\sqrt{d}} + M^U\right) \right]_{1:t, 1:t},$$

where for any matrix A , $A_{1:t, 1:t}$ refers to its top-left t -by- t submatrix.

□

Define the vector $\mathbf{a}_t \in \mathbb{R}^{1 \times L}$ as $[\mathbf{a}_t]_i = (q_t^C (V_t^U)^\top)_i$ if $1 \leq i \leq t$ and $[\mathbf{a}_t]_i = 0$ otherwise.

Denote $\tilde{\mathbf{s}}_t^U = (\mathbf{s}_t^U, \mathbf{0}^{1 \times (L-t)})$. Then, by combining (19) with Lemma C.1, we have

$$\begin{aligned}\tilde{\mathbf{s}}_t^U &= (\mathbf{s}_t^U, \mathbf{0}^{1 \times (L-t)}) = \left(\frac{\mathbf{q}_t^C \mathbf{V}_t^{U\top} \left(\sigma \left(\frac{\mathbf{Q}_t^U \mathbf{K}_t^{U\top}}{\sqrt{d}} + \mathbf{M}_t^U \right) \right)^\top}{\sqrt{d}}, \mathbf{0}^{1 \times (L-t)} \right) \\ &= \frac{\mathbf{a}_t \left(\sigma \left(\frac{\mathbf{Q}_t^U \mathbf{K}_t^{U\top}}{\sqrt{d}} + \mathbf{M}_t^U \right) \right)^\top}{\sqrt{d}}.\end{aligned}$$

Since $\mathbf{V}_t^U$ equals the submatrix which consists of first t rows of $\mathbf{V}^U$, $[\mathbf{a}_t]_i = \left(\mathbf{q}_t^C (\mathbf{V}^U)^\top \right)_i$ for $1 \leq i \leq t$. Then, by stacking $(\mathbf{a}_j)$ together, we have

$$\begin{pmatrix} \mathbf{a}_1 \\ \mathbf{a}_2 \\ \vdots \\ \mathbf{a}_L \end{pmatrix} = (\mathbf{Q}^C \mathbf{V}^{U\top}) \odot \tilde{\mathbf{M}}^C,$$

where $\tilde{\mathbf{M}}^C$ is defined in Theorem 1.1 with $\tilde{\mathbf{M}}_{ij}^C = 1$ if $i \geq j$ and $\tilde{\mathbf{M}}_{ij}^C = 0$ otherwise.

We concatenate $(\tilde{\mathbf{s}}_t^U)_{1 \leq t \leq L}$ in an L -by- L matrix. Then,

$$\begin{aligned}\begin{pmatrix} \tilde{\mathbf{s}}_1^U \\ \tilde{\mathbf{s}}_2^U \\ \vdots \\ \tilde{\mathbf{s}}_L^U \end{pmatrix} &= \frac{1}{\sqrt{d}} \begin{pmatrix} \mathbf{a}_1 \\ \mathbf{a}_2 \\ \vdots \\ \mathbf{a}_L \end{pmatrix} \left(\sigma \left(\frac{\mathbf{Q}^U \mathbf{K}^{U\top}}{\sqrt{d}} + \mathbf{M}^U \right) \right)^\top \\ &= \left(\frac{\mathbf{Q}^C \mathbf{V}^{U\top}}{\sqrt{d}} \odot \tilde{\mathbf{M}}^C \right) \left(\sigma \left(\frac{\mathbf{Q}^U \mathbf{K}^{U\top}}{\sqrt{d}} + \mathbf{M}^U \right) \right)^\top \\ &= \mathbf{S}^U,\end{aligned}\tag{20}$$

where $\mathbf{S}^U$ is given in (10).

The causal-key attention scores $\mathbf{s}_t^C$ in (4) is

$$\mathbf{s}_t^C = \frac{\mathbf{q}_t^C \mathbf{K}_t^{C\top}}{\sqrt{d}}.$$

We also denote $\tilde{\mathbf{s}}_t^C = (\mathbf{s}_t^C, (-\infty)^{1 \times (L-t)})$, where $(-\infty)^{1 \times (L-t)}$ is a $(L-t)$ -dimensional vector with all entries equaling $-\infty$. Then, by concatenating $(\tilde{\mathbf{s}}_t^C)_{1 \leq t \leq L}$ into $\mathbf{S}^C \in \mathbb{R}^{L \times L}$, we have

$$\mathbf{S}^C = \begin{pmatrix} \tilde{\mathbf{s}}_1^C \\ \tilde{\mathbf{s}}_2^C \\ \vdots \\ \tilde{\mathbf{s}}_L^C \end{pmatrix} = \frac{\mathbf{Q}^C \mathbf{K}^{C\top}}{\sqrt{d}} + \mathbf{M}^C.\tag{21}$$

Then, the outputs $\text{Attention}(\mathbf{X}^L)$ satisfies

$$\begin{aligned}
 \text{Attention}(\mathbf{X}^L) &= \begin{pmatrix} \text{attention}(\mathbf{X}^1) \\ \text{attention}(\mathbf{X}^2) \\ \vdots \\ \text{attention}(\mathbf{X}^L) \end{pmatrix} = \begin{pmatrix} \text{softmax}(\mathbf{s}_1^C - \text{SiLU}(\mathbf{s}_1^U)) \mathbf{V}_1^C \\ \text{softmax}(\mathbf{s}_2^C - \text{SiLU}(\mathbf{s}_2^U)) \mathbf{V}_2^C \\ \vdots \\ \text{softmax}(\mathbf{s}_L^C - \text{SiLU}(\mathbf{s}_L^U)) \mathbf{V}_L^C \end{pmatrix} \\
 &= \begin{pmatrix} \text{softmax}(\tilde{\mathbf{s}}_1^C - \text{SiLU}(\tilde{\mathbf{s}}_1^U)) \\ \text{softmax}(\tilde{\mathbf{s}}_2^C - \text{SiLU}(\tilde{\mathbf{s}}_2^U)) \\ \vdots \\ \text{softmax}(\tilde{\mathbf{s}}_L^C - \text{SiLU}(\tilde{\mathbf{s}}_L^U)) \end{pmatrix} \mathbf{V}^C \\
 &= \text{softmax}(\mathbf{S}^C - \text{SiLU}(\mathbf{S}^U)) \mathbf{V}^C \\
 &= \text{softmax}\left(\frac{\mathbf{Q}^C \mathbf{K}^{C\top}}{\sqrt{d}} + \mathbf{M}^C - \text{SiLU}(\mathbf{S}^U)\right) \mathbf{V}^C,
 \end{aligned}$$

where the last inequality above is from (20) and (21).

This completes the proof of Theorem 1.1.

D. Further Details on Multi-Head CASTLE

Forward pass for multi-head CASTLE. Denote $n = n_{\text{head}}$. Given contextualized representations $\mathbf{X}^L$, for each head, we can get $\text{Attention}_i(\mathbf{X}^L)$ as in (11). Then, the outputs of multi-head CASTLE can be obtained by

$$\text{Multihead-Attention}(\mathbf{X}^L) = \text{Concat}\left(\text{Attention}_1(\mathbf{X}^L), \dots, \text{Attention}_n(\mathbf{X}^L)\right) \mathbf{W}^O \in \mathbb{R}^{L \times d}.$$

Parameter count of multi-head CASTLE. In each head, the learnable parameters are $\mathbf{W}_Q^U, \mathbf{W}_K^U, \mathbf{W}_V^U, \mathbf{W}_Q^C, \mathbf{W}_K^C, \mathbf{W}_V^C \in \mathbb{R}^{d_{\text{model}} \times d}$. These parameters sum up to $6n_{\text{head}}dd_{\text{model}}$.

The matrix $\mathbf{W}^O$ has $n_{\text{head}}dd_{\text{model}}$ parameters. Thus, the multi-head CASTLE has $7n_{\text{head}}dd_{\text{model}}$ learnable parameters.

Multi-head CASTLE-SWL has identical formula and parameter counts with multi-head CASTLE and is omitted for clarity.

Parameter count of standard causal attention. The standard causal attention as in (1) has learnable parameters $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V \in \mathbb{R}^{d_{\text{model}} \times d}$ for each head and $\mathbf{W}^O \in \mathbb{R}^{nd \times d_{\text{model}}}$. These parameters sum up to $4n_{\text{head}}dd_{\text{model}}$.

E. Efficient Parallel Training Algorithm and Proof of Theorem 1.2

E.1. Forward Pass

Theorem 1.1 has shown a matrix form of $\text{Attention}(\mathbf{X}^L)$ as in (11). However, computing $\text{Attention}(\mathbf{X}^L)$ directly from (11) still need $O(L^3)$ computational costs because to compute $\mathbf{S}^U$, we need matrix multiplication between L -by- L matrices in (10). In this section, we give an efficient algorithm that enables efficient parallel training and reduces computational complexity to $O(L^2d)$.

We first introduce the notations in this section. We divide the sequence $\{1, \dots, L\}$ into blocks of size B . For simplicity, we assume that L is divisible by B . Let $N = \frac{L}{B}$ and $T_i = \{i * B + 1, \dots, (i + 1) * B\}$ for $0 \leq i \leq N - 1$. Then, $\{1, 2, \dots, L\} = \cup_{i=0}^{N-1} T_i$.

For any matrix $\mathbf{M}$, we use $\mathbf{M}_{T_i, T_j}$ to denote the submatrix whose row and column indexes are in T_i and T_j , respectively. $\mathbf{M}_{T_i, :}$ refers to the submatrix whose row indexes are in T_i . Analogously, $\mathbf{M}_{:, T_j}$ refers to the submatrix whose column indexes are in T_j .

Algorithm 1 Efficient Parallel Forward Pass

Require: $Q^U, K^U, V^U, Q^C, K^C, V^C \in \mathbb{R}^{L \times d}$
Ensure: Output $O \in \mathbb{R}^{L \times d}$

- 1: **Initialization:**
- 2: Initialize $D = \mathbf{0}^{d \times L}$, $O = \mathbf{0}^{L \times d}$, $\ell = \mathbf{0}^{L \times 1}$, and $m = (-\infty)^{L \times 1}$ in HBM
- 3: **Diagonal blocks:**
- 4: **for** $j = 0, \dots, N - 1$ **in parallel do**
- 5: Load $Q_{T_j, :}^C, K_{T_j, :}^C, Q_{T_j, :}^U, K_{T_j, :}^U, V_{T_j, :}^U$ from HBM to on-chip SRAM
- 6: Compute S_{T_j, T_j}^U as in (26)
- 7: Compute A_{T_j, T_j} as in (23)
- 8: Apply online softmax update for block (T_j, T_j) using Algorithm 2
- 9: **end for**
- 10: **Off-diagonal blocks:**
- 11: **for** $k = 1, \dots, N - 1$ **do**
- 12: **for** $j = 0, \dots, N - k - 1$ **in parallel do**
- 13: Load $D_{:, T_j}, Q_{T_j, :}^U, Q_{T_{j+k}, :}^C, K_{T_{j+k-1}, :}^U, K_{T_{j+k}, :}^U, V_{T_{j+k-1}, :}^U, V_{T_{j+k}, :}^U$ from HBM to SRAM
- 14: Update $D_{:, T_j}$ as in (27)
- 15: Compute S_{T_{j+k}, T_j}^U as in (28)
- 16: Write $D_{:, T_j}$ back to HBM
- 17: Compute A_{T_{j+k}, T_j} as in (23)
- 18: Apply online softmax update for block (T_{j+k}, T_j) using Algorithm 2
- 19: **end for**
- 20: **end for**
- 21: $O \leftarrow \text{diag}(\ell)^{-1} O$
- 22: $m \leftarrow m + \log(\ell)$
- 23: Save $m \in \mathbb{R}^{L \times 1}$ and $D \in \mathbb{R}^{d \times L}$ for backward pass
- 24: **return** O

Denote the attention score matrix A by

$$A = \frac{Q^C K^{C\top}}{\sqrt{d}} + M^C - \text{SiLU}(S^U). \quad (22)$$

Then,

$$\text{Attention}(X^L) = \text{softmax}(A) V^C$$

and for any $0 \leq i, j \leq N - 1$, the block A_{T_i, T_j} satisfies

$$A_{T_i, T_j} = \frac{Q_{T_i, :}^C K_{T_j, :}^{C\top}}{\sqrt{d}} + M_{T_i, T_j}^C - \text{SiLU}(S_{T_i, T_j}^U). \quad (23)$$

As in FlashAttention-2 (Dao, 2024), we compute each block of A and apply online softmax (Algorithm 2) on it to obtain $\text{Attention}(X^L)$. The first term $\frac{Q_{T_i, :}^C K_{T_j, :}^{C\top}}{\sqrt{d}}$ in A_{T_i, T_j} as given by (23) can be computed similarly to FlashAttention-2. We mainly focus on the second term $-\text{SiLU}(S_{T_i, T_j}^U)$ which can incur a total computational complexity of $O(L^3 + L^2 d)$ if we apply (10) directly. Notice that as in (10),

$$S^U = \left(\frac{Q^C V^{U\top}}{\sqrt{d}} \odot \widetilde{M}^C \right) \left(\sigma \left(\frac{Q^U K^{U\top}}{\sqrt{d}} + M^U \right) \right)^\top,$$

where the term $\left(\frac{Q^C V^{U\top}}{\sqrt{d}} \odot \widetilde{M}^C \right)$ is a low-rank matrix multiplied by a mask matrix because $Q^C V^{U\top}$ has rank at most d which is normally much smaller than sequence length L . This motivates our algorithm to compute S^U with computational complexity of $O(L^2 d)$ while still enabling parallel training.

Algorithm 2 Online Softmax Update for Block (T_i, T_j)

Require: A_{T_i, T_j} on chip; ℓ , m , O , V^C in HBM

- 1: Load ℓ_{T_i} and m_{T_i} from HBM to on-chip SRAM
 - 2: Compute $m_{T_i}^{\text{new}} = \max(m_{T_i}, \text{row_max}(A_{T_i, T_j}))$
 - 3: Compute $\tilde{P}_{T_i, T_j} = \exp(A_{T_i, T_j} - m_{T_i}^{\text{new}})$
 - 4: Compute $\ell_{T_i}^{\text{new}} = e^{m_{T_i} - m_{T_i}^{\text{new}}} \ell_{T_i} + \text{row_sum}(\tilde{P}_{T_i, T_j})$
 - 5: Update $O_{T_i, :} \leftarrow \text{diag}(e^{m_{T_i} - m_{T_i}^{\text{new}}}) O_{T_i, :} + \tilde{P}_{T_i, T_j} V_{T_j, :}^C$ in HBM
 - 6: Write $\ell_{T_i} \leftarrow \ell_{T_i}^{\text{new}}$ and $m_{T_i} \leftarrow m_{T_i}^{\text{new}}$ to HBM
-

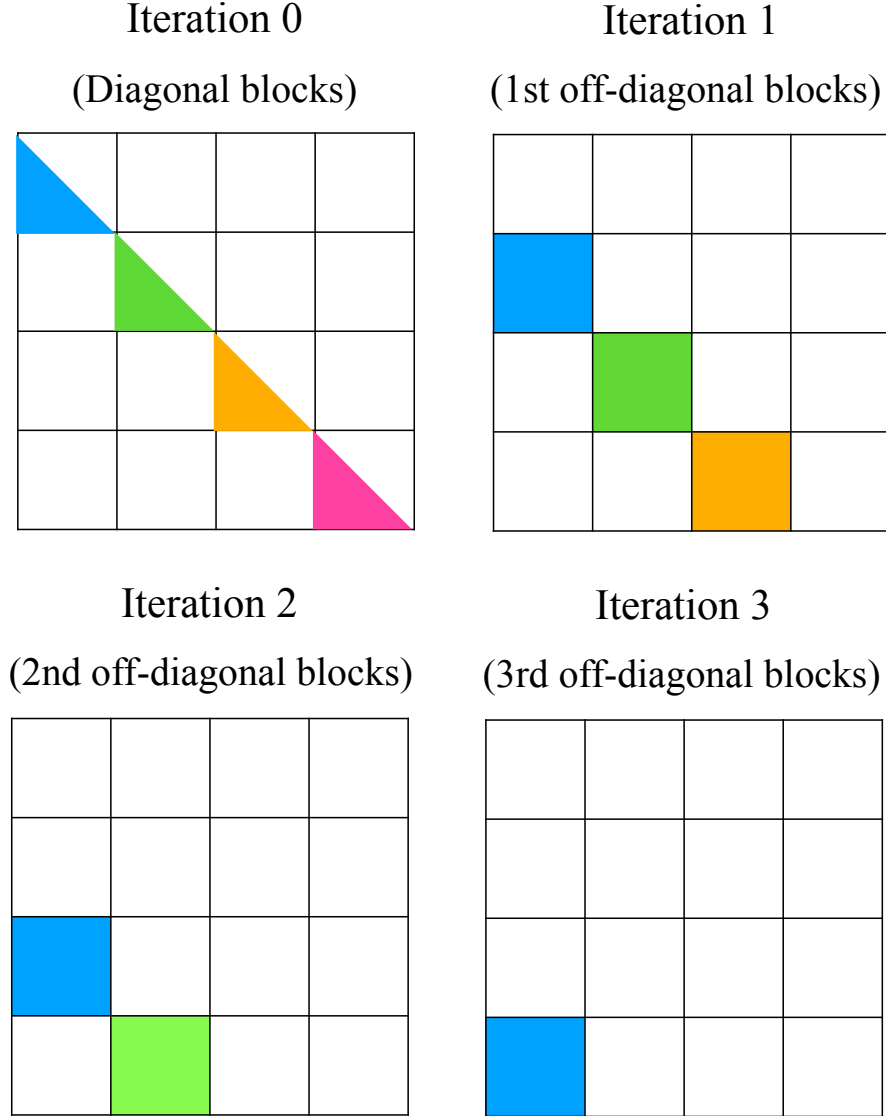


Figure 12. Parallel scheme of Algorithm 1 (forward pass). We begin by computing the diagonal blocks of the attention score matrix A (Iteration 0). In each subsequent iteration k , the k -th off-diagonal blocks are computed. Blocks with different colors represent different kernel instances. In each iteration, each kernel instance is responsible for computing a single block of A and applying online softmax (Algorithm 2) to it. Kernel instances within the same iteration are launched in parallel, while the iterations are executed sequentially. The parallel scheme of Algorithm 3 (backward pass) is the reverse of the forward pass’s parallel scheme.

First, as the upper triangular entries in S^U are all 0, we only need to focus on lower triangular entries in S^U . We divide the

computation of S^U into the following parts

- **diagonal blocks:** S_{T_j, T_j}^U with $0 \leq j \leq N - 1$.
- **k-th off-diagonal blocks:** S_{T_{j+k}, T_j}^U with $0 \leq j \leq N - 1 - k$

By the definition of S^U , we can write S_{T_{j+k}, T_j}^U in a blockwise way as follows

$$\begin{aligned} S_{T_{j+k}, T_j}^U &= Q_{T_{j+k}, :}^C \sum_{i=j}^{j+k-1} \frac{V_{T_i, :}^U}{\sqrt{d}} \left(\sigma \left(\frac{Q_{T_j, :}^U K_{T_i, :}^{U \top}}{\sqrt{d}} + M_{T_j, T_i}^U \right) \right)^\top \\ &+ \left(\frac{Q_{T_{j+k}, :}^C V_{T_{j+k}, :}^U}{\sqrt{d}} \odot \widetilde{M}_{T_{j+k}, T_{j+k}}^C \right) \left(\sigma \left(\frac{Q_{T_j, :}^U K_{T_{j+k}, :}^{U \top}}{\sqrt{d}} + M_{T_j, T_{j+k}}^U \right) \right)^\top. \end{aligned} \quad (24)$$

However, computing each S_{T_{j+k}, T_j}^U this way will lead to $O(L^3)$ computational cost in total. We need more efficient way to reduce training costs.

The key observation in the development of this efficient parallel training algorithm is that the matrix $(Q^C V^{U \top}) \odot \widetilde{M}^C$ is a low-rank matrix multiplied by a mask because $Q^C V^{U \top}$ has rank equal to or less than the head dimension d rather than the sequence length L . This enables us to compute S^U defined in (10) with lower computational costs. More specifically, we will rely on the following auxiliary variable to reduce computational costs.

We first init an auxiliary variable $D^{(0)} = \mathbf{0}^{d \times L}$. Then, when computing the k -th off-diagonal blocks, we will maintain the following relation that

$$D_{:, T_j}^{(k)} = \sum_{i=j}^{j+k-1} V_{T_i, :}^U \left(\sigma \left(\frac{Q_{T_j, :}^U K_{T_i, :}^{U \top}}{\sqrt{d}} + M_{T_j, T_i}^U \right) \right)^\top. \quad (25)$$

We will compute the blocks of S^U in the following order: diagonal blocks, 1st off-diagonal blocks, 2nd off-diagonal blocks, $\dots$, $(N - 1)$ -th off-diagonal blocks.

Diagonal blocks of S^U . For any $0 \leq j \leq N - 1$,

$$S_{T_j, T_j}^U = \left(\frac{Q_{T_j, :}^C V_{T_j, :}^U}{\sqrt{d}} \odot \widetilde{M}_{T_j, T_j}^C \right) \left(\sigma \left(\frac{Q_{T_j, :}^U K_{T_j, :}^{U \top}}{\sqrt{d}} + M_{T_j, T_j}^U \right) \right)^\top. \quad (26)$$

1st off-diagonal blocks of S^U . For any $0 \leq j \leq N - 2$, update $D_{:, T_j}^{(1)}$ as

$$D_{:, T_j}^{(1)} = V_{T_j, :}^U \left(\sigma \left(\frac{Q_{T_j, :}^U K_{T_j, :}^{U \top}}{\sqrt{d}} + M_{T_j, T_j}^U \right) \right)^\top$$

This satisfies (25) with $k = 1$. Then, it follows from (24) that

$$\begin{aligned} S_{T_{j+1}, T_j}^U &= \frac{Q_{T_{j+1}, :}^C D_{:, T_j}^{(1)}}{\sqrt{d}} \\ &+ \left(\frac{Q_{T_{j+1}, :}^C V_{T_{j+1}, :}^U}{\sqrt{d}} \odot \widetilde{M}_{T_{j+1}, T_{j+1}}^C \right) \left(\sigma \left(\frac{Q_{T_j, :}^U K_{T_{j+1}, :}^{U \top}}{\sqrt{d}} + M_{T_j, T_{j+1}}^U \right) \right)^\top. \end{aligned}$$

The k -th off-diagonal blocks of S^U . If we have already computed the $(k-1)$ -th off-diagonal blocks and $D^{(k-1)}$ satisfies (25) with $k-1$, the k -th off-diagonal blocks can be computed in the following way. First, we update $D^{(k)}$ as follows

$$D_{:,T_j}^{(k)} = D_{:,T_j}^{(k-1)} + V_{T_{j+k-1},:}^U \left(\sigma \left(\frac{Q_{T_j,:}^U K_{T_{j+k-1},:}^U}{\sqrt{d}} + M_{T_j,T_{j+k-1}}^U \right) \right)^\top. \quad (27)$$

By induction hypothesis (25), $D_{:,T_j}^{(k)}$ also satisfies (25) with k . Then, it follows by (24) that

$$\begin{aligned} S_{T_{j+k},T_j}^U &= \frac{Q_{T_{j+k},:}^C D_{:,T_j}^{(k)}}{\sqrt{d}} \\ &+ \left(\frac{Q_{T_{j+k},:}^C V_{T_{j+k},:}^U}{\sqrt{d}} \odot \widetilde{M}_{T_{j+k},T_{j+k}}^C \right) \left(\sigma \left(\frac{Q_{T_j,:}^U K_{T_{j+k},:}^U}{\sqrt{d}} + M_{T_j,T_{j+k}}^U \right) \right)^\top. \end{aligned} \quad (28)$$

We can compute all k -th off-diagonal blocks in the above way. When computing each S_{T_{j+k},T_j}^U , we only need to update $D^{(k)}$ and then compute S_{T_{j+k},T_j}^U as (28). Both (27) and (28) take $O(L^2 d)$ FLOPs.

Next, we describe the design of a parallel algorithm. The parallelization scheme must satisfy the following requirements:

- S_{T_{j+k},T_j}^U must be computed after S_{T_{j+k-1},T_j}^U because computing S_{T_{j+k},T_j}^U in (28) requires $D^{(k)}$ which is derived by updating $D^{(k-1)}$ in (27). Therefore, A_{T_{j+k},T_j} should be computed after A_{T_{j+k-1},T_j} .
- To ensure correctness, online softmax cannot be applied simultaneously to A_{T_i,T_j} and A_{T_i,T_k} for $j \neq k$.

To meet these constraints, we launch kernel instances to compute attention outputs with respect to blocks in the following order: starting with the diagonal blocks, followed by the first off-diagonal blocks, then the second, and so on up to the $(N-1)$ -th off-diagonal blocks. This parallel execution strategy for Algorithm 1 is illustrated in Figure 12.

As we can see, we launch one kernel instance for each block S_{T_i,T_j} with $i \geq j$. Thus, there are

$$\frac{L}{B} \cdot \left(\frac{L}{B} - 1 \right) = O\left(\frac{L^2}{B^2}\right) \text{ block.}$$

As in Algorithm 1, inside each block, we only have $O(1)$ matrix multiplications between B -by- d matrices or B -by- B matrices, where B is the block size. Other operations are entrywise. In addition, we choose $B = O(d)$. (In practice, we found that $B = 64$ optimizes the performance of our training kernels.) Thus, each block has only $O(B^2 d + B^3) = O(B^2 d)$ FLOPs. Thus, the computational complexity is

$$O\left(\frac{L^2}{B^2}\right) \cdot O(B^2 d) = L^2 d.$$

It is straightforward that the space complexity is $O(Ld)$.

E.2. Backward Pass

In this section, we introduce the backward pass algorithm for efficient parallel training. It is mainly derived by reversing the forward pass.

Recall that in the forward pass, we compute the blocks in the following order: diagonal blocks, 1st off-diagonal blocks, 2nd off-diagonal blocks, $\dots$, $(N-1)$ -th off-diagonal blocks. Then, in the backward pass, we compute the gradients in the inverse order as follows: $(N-1)$ -th off-diagonal blocks, $(N-2)$ -th off-diagonal blocks, $\dots$, 1-st off-diagonal blocks, diagonal blocks.

Algorithm 3 Efficient Parallel Backward Pass

Require: $\mathbf{dO} \in \mathbb{R}^{L \times d_{\text{model}}}$

Ensure: Gradients $\mathbf{dQ}^U, \mathbf{dK}^U, \mathbf{dV}^U, \mathbf{dQ}^C, \mathbf{dK}^C, \mathbf{dV}^C \in \mathbb{R}^{L \times d}$

- 1: **Initialization:**
- 2: Initialize $\mathbf{dD} = \mathbf{0}^{d \times L}, \mathbf{dQ}^U = \mathbf{0}^{L \times d}, \mathbf{dK}^U = \mathbf{0}^{L \times d}, \mathbf{dV}^U = \mathbf{0}^{L \times d}, \mathbf{dQ}^C = \mathbf{0}^{L \times d}, \mathbf{dK}^C = \mathbf{0}^{L \times d}, \mathbf{dV}^C = \mathbf{0}^{L \times d}$
- 3: Let $\mathbf{m} \in \mathbb{R}^{L \times 1}$ and $\mathbf{D} \in \mathbb{R}^{d \times L}$ be saved from the forward pass (Algorithm 1)
- 4: **Preprocess:**
- 5: Compute $\Delta \in \mathbb{R}^{L \times 1}$ with $\Delta_i = \mathbf{dO}_{i,:}^\top \mathbf{O}_{i,:}$
- 6: **Off-diagonal blocks:**
- 7: **for** $k = N - 1, \dots, 1$ **do**
- 8: **for** $j = 0, \dots, N - k - 1$ **in parallel do**
- 9: Compute $\mathbf{S}_{T_{j+k}, T_j}^U$ as in (28) and $\mathbf{A}_{T_{j+k}, T_j}$ as in (23)
- 10: Compute $\mathbf{P}_{T_{j+k}, T_j}$ and $\mathbf{dP}_{T_{j+k}, T_j}$ as in (29) and (30)
- 11: Compute $\mathbf{dS}_{T_{j+k}, T_j}^C$ and $\mathbf{dS}_{T_{j+k}, T_j}^U$ as in (32) and (33)
- 12: Update $\mathbf{dV}_{T_j,:}^C, \mathbf{dQ}_{T_j,:}^C, \mathbf{dK}_{T_j,:}^C$ as in (31) and (34)
- 13: Update $\mathbf{dQ}_{T_{j+k},:}^C, \mathbf{dD}_{:,T_j}^{(k)}, \mathbf{dV}_{T_{j+k},:}^U, \mathbf{dQ}_{T_j,:}^U, \mathbf{dK}_{T_{j+k},:}^U$ as in (35)
- 14: Update $\mathbf{dV}_{T_{j+k-1},:}^U, \mathbf{dQ}_{T_j,:}^U, \mathbf{dK}_{T_{j+k-1},:}^U$ as in (36)
- 15: Compute $\mathbf{D}_{:,T_j}^{(k-1)}$ as in (37)
- 16: **end for**
- 17: **end for**
- 18: **Diagonal blocks:**
- 19: **for** $j = 0, \dots, N - 1$ **in parallel do**
- 20: Compute $\mathbf{S}_{T_j, T_j}^U$ as in (26) and $\mathbf{A}_{T_j, T_j}$ as in (23)
- 21: Compute $\mathbf{P}_{T_j, T_j}$ and $\mathbf{dP}_{T_j, T_j}$ as in (38) and (39)
- 22: Compute $\mathbf{dS}_{T_j, T_j}^C$ and $\mathbf{dS}_{T_j, T_j}^U$ as in (41) and (42)
- 23: Update $\mathbf{dV}_{T_j,:}^C, \mathbf{dQ}_{T_j,:}^C, \mathbf{dV}_{T_j,:}^U, \mathbf{dQ}_{T_j,:}^U, \mathbf{dK}_{T_j,:}^U$ as in (40) and (43)
- 24: **end for**
- 25: **return** $\mathbf{dQ}^U, \mathbf{dK}^U, \mathbf{dV}^U, \mathbf{dQ}^C, \mathbf{dK}^C, \mathbf{dV}^C$

As in (Dao et al., 2022), we first preprocess $\Delta \in \mathbb{R}^{L \times 1}$ with

$$\Delta_i = \mathbf{dO}_{i,:}^\top \mathbf{O}_{i,:}.$$

Let $\mathbf{D}^{(N-1)}$ be the $\mathbf{D}$ saved for backward pass in Algorithm 1. Also, let $\mathbf{m}$ be the vector $\mathbf{m}$ saved for backward pass in Algorithm 1. Set $\mathbf{dD}^{(N-1)} = \mathbf{0}^{d \times L}$.

Then, we iterate over $k = N - 1, \dots, 1$. In each iteration, we compute the corresponding gradients and update $\mathbf{D}^{(k)}$ to $\mathbf{D}^{(k-1)}$ inversely as in the forward pass. Thus, for each k , before we launch kernels for the k -th off-diagonal blocks, we already have $\mathbf{D}^{(k)}$.

k -th off-diagonal blocks. For any $0 \leq j \leq N - 1 - k$, we first compute $\mathbf{S}_{T_{j+k}, T_j}^U$ as in (28) and $\mathbf{A}_{T_{j+k}, T_j}$ as in (23). Recall that in the end of the forward pass (Algorithm 1), we have set $\mathbf{m} \leftarrow \mathbf{m} + \log(\ell)$. Then, the attention weights can be computed by

$$\mathbf{P}_{T_{j+k}, T_j} = \exp(\mathbf{A}_{T_{j+k}, T_j} - \mathbf{m}_{T_{j+k}}). \quad (29)$$

Compute the gradient of attention weights

$$\mathbf{dP}_{T_{j+k}, T_j} = \mathbf{dO}_{T_{j+k},:} \mathbf{V}_{T_j,:}^\top. \quad (30)$$

Then, update the gradient of $\mathbf{V}_{T_j,:}^C$ as follows

$$\mathbf{dV}_{T_j,:}^C \leftarrow \mathbf{dV}_{T_j,:}^C + \mathbf{P}_{T_{j+k}, T_j}^\top \mathbf{dO}_{T_{j+k},:}. \quad (31)$$

Then, compute the gradients of attention weights

$$dS_{T_{j+k}, T_j}^C = P_{T_{j+k}, T_j} \odot (dP_{T_{j+k}, T_j} - \Delta_{T_{j+k}}) \quad (32)$$

and

$$dS_{T_{j+k}, T_j}^U = -\nabla \text{SiLU}(S_{T_{j+k}, T_j}^U) \odot P_{T_{j+k}, T_j} \odot (dP_{T_{j+k}, T_j} - \Delta_{T_{j+k}}). \quad (33)$$

Then, we update the gradients of $Q_{T_{j+k},:}^C, K_{T_j,:}^C$ through the backward pass of (23) as follows

$$\begin{aligned} dQ_{T_{j+k},:}^C &\leftarrow dQ_{T_{j+k},:}^C + \frac{dS_{T_{j+k}, T_j}^C K_{T_j,:}^C}{\sqrt{d}}, \\ dK_{T_j,:}^C &\leftarrow dK_{T_j,:}^C + \frac{dS_{T_{j+k}, T_j}^C{}^\top Q_{T_{j+k},:}^C}{\sqrt{d}}. \end{aligned} \quad (34)$$

Then, we update the gradients of $Q_{T_{j+k},:}^C, D_{:,T_j}^{(k)}, V_{T_{j+k},:}^U, Q_{T_j,:}^U, K_{T_{j+k},:}^U$ through the backward pass of (28) as follows

$$\begin{aligned} dQ_{T_{j+k},:}^C &\leftarrow dQ_{T_{j+k},:}^C + \frac{dS_{T_{j+k}, T_j}^U D_{:,T_j}^{(k)^\top}}{\sqrt{d}} + \frac{E V_{T_{j+k},:}^U}{\sqrt{d}} \\ dD_{:,T_j}^{(k)} &\leftarrow dD_{:,T_j}^{(k)} + \frac{Q_{T_{j+k},:}^C{}^\top dS_{T_{j+k}, T_j}^U}{\sqrt{d}} \\ dV_{T_{j+k},:}^U &\leftarrow dV_{T_{j+k},:}^U + \frac{E^\top Q_{T_{j+k},:}^C}{\sqrt{d}} \\ dQ_{T_j,:}^U &\leftarrow dQ_{T_j,:}^U + \frac{F K_{T_{j+k},:}^U}{\sqrt{d}} \\ dK_{T_{j+k},:}^U &\leftarrow dK_{T_{j+k},:}^U + \frac{F^\top Q_{T_j,:}^U}{\sqrt{d}}, \end{aligned} \quad (35)$$

where auxiliary matrices

$$\begin{aligned} E &= \left(dS_{T_{j+k}, T_j}^U \sigma \left(\frac{Q_{T_j,:}^U K_{T_{j+k},:}^U{}^\top}{\sqrt{d}} + M_{T_j, T_{j+k}}^U \right) \right) \odot \widetilde{M}_{T_{j+k}, T_{j+k}}^C, \\ F &= \left(\left(dS_{T_{j+k}, T_j}^U \right)^\top \left(\frac{Q_{T_{j+k},:}^C V_{T_{j+k},:}^U{}^\top}{\sqrt{d}} \odot \widetilde{M}_{T_{j+k}, T_{j+k}}^C \right) \right) \\ &\quad \odot \nabla \sigma \left(\frac{Q_{T_j,:}^U K_{T_{j+k},:}^U{}^\top}{\sqrt{d}} + M_{T_j, T_{j+k}}^U \right). \end{aligned}$$

Then, we update the gradients of $V_{T_{j+k-1},:}^U, Q_{T_j,:}^U, K_{T_{j+k-1},:}^U$ from the backward pass of (27) as follows

$$\begin{aligned} dV_{T_{j+k-1},:}^U &\leftarrow dV_{T_{j+k-1},:}^U \\ &\quad + \left(\sigma \left(\frac{Q_{T_j,:}^U K_{T_{j+k-1},:}^U{}^\top}{\sqrt{d}} + M_{T_j, T_{j+k-1}}^U \right) \right)^\top (dD_{:,T_j}^{(k)})^\top \\ dQ_{T_j,:}^U &\leftarrow dQ_{T_j,:}^U + \frac{G K_{T_{j+k-1},:}^U}{\sqrt{d}} \\ dK_{T_{j+k-1},:}^U &\leftarrow dK_{T_{j+k-1},:}^U + \frac{G^\top Q_{T_j,:}^U}{\sqrt{d}} \end{aligned} \quad (36)$$

where the auxiliary matrix

$$\mathbf{G} = \left(\left(\mathbf{d} \mathbf{D}_{:,T_j}^{(k)} \right)^\top \mathbf{V}_{T_j+k-1,:}^U \right)^\top \odot \nabla \sigma \left(\frac{\mathbf{Q}_{T_j,:}^U \mathbf{K}_{T_j+k-1,:}^U}{\sqrt{d}} + \mathbf{M}_{T_j,T_j+k-1}^U \right).$$

After updating gradients, we set $\mathbf{d} \mathbf{D}_{:,T_j}^{(k-1)} \leftarrow \mathbf{d} \mathbf{D}_{:,T_j}^{(k)}$ and get $\mathbf{D}_{:,T_j}^{(k-1)}$ back from $\mathbf{D}_{:,T_j}^{(k)}$ by reversing (27) as follows

$$\mathbf{D}_{:,T_j}^{(k-1)} = \mathbf{D}_{:,T_j}^{(k)} - \mathbf{V}_{T_j+k-1,:}^U \left(\sigma \left(\frac{\mathbf{Q}_{T_j,:}^U \mathbf{K}_{T_j+k-1,:}^U}{\sqrt{d}} + \mathbf{M}_{T_j,T_j+k-1}^U \right) \right)^\top. \quad (37)$$

Diagonal blocks. For any $0 \leq j \leq N-1$, we first compute $\mathbf{S}_{T_j,T_j}^U$ as in (26) and $\mathbf{A}_{T_j,T_j}$ as in (23). Then, compute the attention weights

$$\mathbf{P}_{T_j,T_j} = \exp(\mathbf{A}_{T_j,T_j} - \mathbf{m}_{T_j}). \quad (38)$$

Compute the gradient of attention weights

$$\mathbf{d} \mathbf{P}_{T_j,T_j} = \mathbf{d} \mathbf{O}_{T_j,:} \mathbf{V}_{T_j,:}^\top \quad (39)$$

Then, update the gradient of $\mathbf{V}_{T_j,:}^C$ as follows

$$\mathbf{d} \mathbf{V}_{T_j,:}^C \leftarrow \mathbf{d} \mathbf{V}_{T_j,:}^C + \mathbf{P}_{T_j,T_j}^\top \mathbf{d} \mathbf{O}_{T_j,:} \quad (40)$$

Then, compute the gradients of attention weights

$$\mathbf{d} \mathbf{S}_{T_j,T_j}^C = \mathbf{P}_{T_j,T_j} \odot (\mathbf{d} \mathbf{P}_{T_j,T_j} - \Delta_{T_j}) \quad (41)$$

and

$$\mathbf{d} \mathbf{S}_{T_j,T_j}^U = -\nabla \text{SiLU}(\mathbf{S}_{T_j,T_j}^U) \odot \mathbf{P}_{T_j,T_j} \odot (\mathbf{d} \mathbf{P}_{T_j,T_j} - \Delta_{T_j}). \quad (42)$$

Then, we update the gradients of $\mathbf{Q}_{T_j,:}^C$, $\mathbf{V}_{T_j,:}^U$, $\mathbf{Q}_{T_j,:}^U$, $\mathbf{K}_{T_j,:}^U$ through the backward pass of (26) as follows

$$\begin{aligned} \mathbf{d} \mathbf{Q}_{T_j,:}^C &\leftarrow \mathbf{d} \mathbf{Q}_{T_j,:}^C + \frac{\mathbf{E} \mathbf{V}_{T_j,:}^U}{\sqrt{d}}, \\ \mathbf{d} \mathbf{V}_{T_j,:}^U &\leftarrow \mathbf{d} \mathbf{V}_{T_j,:}^U + \frac{\mathbf{E}^\top \mathbf{Q}_{T_j,:}^C}{\sqrt{d}}, \\ \mathbf{d} \mathbf{Q}_{T_j,:}^U &\leftarrow \mathbf{d} \mathbf{Q}_{T_j,:}^U + \frac{\mathbf{F} \mathbf{K}_{T_j,:}^U}{\sqrt{d}}, \\ \mathbf{d} \mathbf{K}_{T_j,:}^U &\leftarrow \mathbf{d} \mathbf{K}_{T_j,:}^U + \frac{\mathbf{F}^\top \mathbf{Q}_{T_j,:}^U}{\sqrt{d}}, \end{aligned} \quad (43)$$

where auxiliary matrices

$$\begin{aligned} \mathbf{E} &= \left(\mathbf{d} \mathbf{S}_{T_j,T_j}^U \sigma \left(\frac{\mathbf{Q}_{T_j,:}^U \mathbf{K}_{T_j,:}^U}{\sqrt{d}} + \mathbf{M}_{T_j,T_j}^U \right) \right) \odot \widetilde{\mathbf{M}}_{T_j,T_j}^C, \\ \mathbf{F} &= \left(\left(\mathbf{d} \mathbf{S}_{T_j,T_j}^U \right)^\top \left(\frac{\mathbf{Q}_{T_j,:}^C \mathbf{V}_{T_j,:}^U}{\sqrt{d}} \odot \widetilde{\mathbf{M}}_{T_j,T_j}^C \right) \right) \odot \nabla \sigma \left(\frac{\mathbf{Q}_{T_j,:}^U \mathbf{K}_{T_j,:}^U}{\sqrt{d}} + \mathbf{M}_{T_j,T_j}^U \right). \end{aligned}$$

Then, the pseudo-code of backward pass is illustrated in Algorithm 3. The backward pass's parallel scheme is naturally the reverse of the forward pass's parallel scheme.

We remark that when computing the gradients with respect to the block (T_{j+k}, T_j) , we update both $\mathbf{V}_{T_{j+k},:}^U$ and $\mathbf{V}_{T_{j+k-1},:}^U$. Consequently, the block $\mathbf{V}_{T_{j+k-1},:}^U$ receives contributions from two sources: (T_{j+k}, T_j) and (T_{j+k-1}, T_{j-1}) . To prevent overlapping updates, we introduce two auxiliary variables for storing intermediate results in $d\mathbf{V}^U$, namely $d\widehat{\mathbf{V}}^U$ and $d\widetilde{\mathbf{V}}^U$. Specifically, in block (T_{j+k}, T_j) , the update to $d\mathbf{V}_{T_{j+k},:}^U$ is accumulated in $d\widehat{\mathbf{V}}^U$, while the update to $d\mathbf{V}_{T_{j+k-1},:}^U$ is accumulated in $d\widetilde{\mathbf{V}}^U$. After all gradients with respect to off-diagonal blocks have been computed, we obtain the true gradient of $d\mathbf{V}^U$ by $d\mathbf{V}^U \leftarrow d\widehat{\mathbf{V}}^U + d\widetilde{\mathbf{V}}^U$. The same procedure is applied to $d\mathbf{K}^U$.

Similar to the forward pass, we launch one kernel instance for each block $\mathbf{S}_{T_i, T_j}$ with $i \geq j$. Thus, there are

$$\frac{L}{B} \cdot \left(\frac{L}{B} - 1 \right) = O\left(\frac{L^2}{B^2}\right) \text{ block.}$$

Inside each block, as in Algorithm 3, we only have $O(1)$ matrix multiplications between B -by- d matrices or B -by- B matrices, where B is the block size. And other operations are entrywise. In practice, we always choose $B = O(d)$. (We found that $B = 64$ optimizes the performance of our training kernels.) Thus, each block has only $O(B^2d + B^3) = O(B^2d)$ FLOPs. Thus, the computational complexity is

$$O\left(\frac{L^2}{B^2}\right) \cdot O(B^2d) = L^2d.$$

It is also straightforward that the space complexity is $O(Ld)$.

F. Efficient Inference with UQ-KV Cache

In this section, we present a detailed description of CASTLE’s inference algorithm. Because the prefilling stage requires computing the UQ–KV cache, we first motivate the need for this cache by outlining the decoding process in Appendix F.1. Appendix F.2 compares the size of the UQ–KV cache in CASTLE with the standard KV cache used in causal attention. Appendix F.3 then provides the prefilling algorithm.

F.1. Decoding Algorithm

Throughout this section, we denote head dimension by d , the number of heads by $n_{\text{castle.head}}$. We fix one t and consider the decoding process when generating the $(t+1)$ -th token.

Decoding Algorithm consists of 2 steps: (1) rank-1 updating step and (2) combining step.

Rank-1 Updating Step. Directly computing U^t from (2) at each position will lead to $\Omega(L^3d)$ computational costs in total, which is prohibitive. Fortunately, we can instead compute U^t from U^{t-1} through a rank-1 update as in (12) as follows

$$U^t = \left(U^{t-1} + \sigma \left(\frac{\mathbf{Q}_{t-1}^U \mathbf{k}_t^{U^\top}}{\sqrt{d}} + [\mathbf{M}_t^U]_{1:t-1, t} \right) \mathbf{v}_t^U \right).$$

We prove the equivalence between 2 and (12) as follows.

Proof of (12). By (2),

$$\mathbf{u}_s^t = \sum_{j=1}^t \sigma \left(\frac{\mathbf{q}_s^U \mathbf{k}_j^{U^\top}}{\sqrt{d}} + [\mathbf{M}_t^U]_{sj} \right) \mathbf{v}_j^U.$$

Thus, for $1 \leq s < t$,

$$\mathbf{u}_s^t - \mathbf{u}_s^{t-1} = \sigma \left(\frac{\mathbf{q}_s^U \mathbf{k}_t^{U^\top}}{\sqrt{d}} + [\mathbf{M}_t^U]_{st} \right) \mathbf{v}_t^U.$$

Since $[\mathbf{M}_t^U]_{t,t} = -\infty$, the last row of U^t is all-zeros. This yields (12). $\square$

This step is called updating step since we are getting U^t by rank-1 update from U^{t-1} .

To compute (12) efficiently, obviously, we need to store U^{t-1} from the last step, as we need to compute U^t from U^{t-1} . The U-cache from the previous step has the following size:

- U-cache stored from previous step: $[B, t-1, n_{\text{castle.head}}, d]$.

In rank-1 updating step, there is one matrix-vector multiplication: $Q_{t-1}^U k_t^{U^\top}$. Note that $[M_t^U]_{1:t-1,t}$ is the subvector of the last column of $M_t^U \in \mathbb{R}^{t \times t}$ defined in (3) for CASTLE and (8) for CASTLE-SWL. We have slightly different ways to deal with this matrix-vector multiplication in CASTLE and CASTLE-SWL because M_t^U is different in CASTLE and CASTLE-SWL. This leads to Q-cache in CASTLE and CASTLE-SWL having different sizes.

Q-cache in CASTLE. By the definition of M_t^U in (3), $[M_t^U]_{1:t-1,t} \in \mathbb{R}^{(t-1) \times 1}$ is an all-zeros vector, thus, by storing the entire Q_{t-1}^U , we can compute $Q_{t-1}^U k_t^U$ directly without computing $Q_{t-1}^U = X^{t-1} W_Q^U$. Therefore, in CASTLE, Q-cache from the previous step has the following size

- Q-cache stored from previous step in CASTLE: $[B, t-1, n_{\text{castle.head}}, d]$.

Q-cache in CASTLE-SWL. By the definition of M_t^U in (8) (an illustration can be found in Figure 14), the last W entries in $[M_t^U]_{1:t-1,t}$ are all 0, and the other entries are all $-\infty$. Thus, in the vector $\sigma\left(\frac{Q_{t-1}^U k_t^{U^\top}}{\sqrt{d}} + [M_t^U]_{1:t-1,t}\right) \in \mathbb{R}^{(t-1) \times 1}$, its s -th entry is 0 if $s < t - W$, where W is the window size. Therefore, we only need to store the last W rows in Q_{t-1}^U . In CASTLE-SWL, the Q-cache from the previous step is of the following size:

- Q-cache stored from previous step in CASTLE-SWL: $[B, (t-1) \wedge W, n_{\text{castle.head}}, d]$.

Then, we can get U^t by rank-1 update from U^{t-1} using the column vector $\sigma\left(\frac{Q_{t-1}^U k_t^{U^\top}}{\sqrt{d}} + [M_t^U]_{:,t}\right) \in \mathbb{R}^{(t-1) \times 1}$ and the row vector $v_t^U \in \mathbb{R}^{1 \times d}$. This completes the rank-1 update step.

Combining step. Next, since we already have U^t , we can compute the output by going through (4), (5), (6) and (7) sequentially. Then, by the same reason with using KV cache in standard causal attention, we need to use K_{t-1}^C and V_{t-1}^C stored from the previous step which are of sizes

- K-cache stored from previous step: $[B, t-1, n_{\text{castle.head}}, d]$,
- V-cache stored from previous step: $[B, t-1, n_{\text{castle.head}}, d]$.

After computing the output, we need to renew UQ-KV cache to reduce computational costs for the next step. This yields the size of UQ-KV cache:

- U-cache size: $[B, t, n_{\text{castle.head}}, d]$,
- Q-cache size:
 - In CASTLE: $[B, t, n_{\text{castle.head}}, d]$
 - In CASTLE-SWL: $[B, t \wedge W, n_{\text{castle.head}}, d]$ (W is the window size)
- K-cache size: $[B, t, n_{\text{castle.head}}, d]$,
- V-cache size: $[B, t, n_{\text{castle.head}}, d]$,

In practice, cache renewal is incorporated into the combining step. The pseudocode of the process described above is in Algorithm 4.

Algorithm 4 Decoding Algorithm (Generating the $(t + 1)$ -th Token)

Require: Representation $\mathbf{x}_t \in \mathbb{R}^{1 \times d_{\text{model}}}$; UQ-KV cache $\mathbf{U}^{t-1}, \mathbf{Q}_{t-1}^U, \mathbf{K}_{t-1}^C, \mathbf{V}_{t-1}^C \in \mathbb{R}^{(t-1) \times d}$

Ensure: Output $\mathbf{o}_t \in \mathbb{R}^{1 \times d}$ and updated cache $\mathbf{U}^t, \mathbf{Q}_t^U, \mathbf{K}_t^C, \mathbf{V}_t^C \in \mathbb{R}^{t \times d}$

- 1: Compute $\mathbf{q}_t^U = \mathbf{x}_t \mathbf{W}_Q^U, \mathbf{k}_t^U = \mathbf{x}_t \mathbf{W}_K^U, \mathbf{v}_t^U = \mathbf{x}_t \mathbf{W}_V^U$
 - 2: Compute $\mathbf{q}_t^C = \mathbf{x}_t \mathbf{W}_Q^C, \mathbf{k}_t^C = \mathbf{x}_t \mathbf{W}_K^C, \mathbf{v}_t^C = \mathbf{x}_t \mathbf{W}_V^C$
 - 3: **Rank-1 update:**
 - 4: Update $\mathbf{U}^t$ as in (12)
 - 5: **Combining step:**
 - 6: Update $\mathbf{Q}_t^U \leftarrow [\mathbf{Q}_{t-1}^U; \mathbf{q}_t^U]$
 - 7: Update $\mathbf{K}_t^C \leftarrow [\mathbf{K}_{t-1}^C; \mathbf{k}_t^C]$
 - 8: Update $\mathbf{V}_t^C \leftarrow [\mathbf{V}_{t-1}^C; \mathbf{v}_t^C]$
 - 9: Compute $\mathbf{s}_t^C = \frac{\mathbf{q}_t^C (\mathbf{K}_t^C)^\top}{\sqrt{d}} \in \mathbb{R}^{1 \times t}$ as in (4)
 - 10: Compute $\mathbf{s}_t^U = \frac{\mathbf{q}_t^U (\mathbf{U}^t)^\top}{\sqrt{d}} \in \mathbb{R}^{1 \times t}$ as in (5)
 - 11: Compute $\mathbf{p}_t = \text{softmax}(\mathbf{s}_t^C - \text{SiLU}(\mathbf{s}_t^U)) \in \mathbb{R}^{1 \times t}$ as in (6)
 - 12: Compute $\mathbf{o}_t = \mathbf{p}_t \mathbf{V}_t^C \in \mathbb{R}^{1 \times d}$ as in (7)
 - 13: **return** $\mathbf{o}_t, \mathbf{U}^t, \mathbf{Q}_t^U, \mathbf{K}_t^C, \mathbf{V}_t^C$
-

F.2. Memory Requirement of UQ-KV Cache

In this section, we compare the memory usage of the UQ-KV cache in CASTLE with that of the standard KV cache used in causal attention. We use Baseline-XL and CASTLE(-SWL)-XL as an example.

Denote the number of heads in baseline model and CASTLE by n_{head} and $n_{\text{castle.head}}$, respectively. Denote batch size, sequence length and head dimension by B, T and d .

In practice, we use Bfloat16 to store KV cache in baseline models and UQ-KV cache in CASTLE and CASTLE-SWL.

As in Table 4, we set $n_{\text{castle.head}} = \frac{9}{16} n_{\text{head}}$ to align the number of parameters.

KV cache in Standard Causal Attention. Both K cache and V cache are of size $[B, T, n_{\text{head}}, d]$. Then, the total memory requirement is

$$2 * B * T * n_{\text{head}} * 2 \text{ Byte} = 4 BT n_{\text{head}} d \text{ Bytes.}$$

UQ-KV cache in CASTLE. By the analysis in Appendix F.1, UQ-KV cache requires memory

$$4 * B * T * n_{\text{castle.head}} * d * 2 \text{ Bytes} = 8 BT n_{\text{castle.head}} d \text{ Bytes} = \frac{9}{2} BT n_{\text{head}} d \text{ Bytes.}$$

UQ-KV cache in CASTLE-SWL. By the analysis in Appendix F.1, UQ-KV cahce in CASTLE requires memory

$$3 * B * T * n_{\text{castle.head}} * d * 2 \text{ Bytes} + T \wedge W * B * n_{\text{castle.head}} * d * 2 \text{ Bytes.}$$

- When $t \leq W$ (window size), UQ-KV cache in CASTLE-SWL uses $\frac{9}{2} BT n_{\text{head}} d$ Bytes.
- When $t \gg W$, UQ-KV cache uses $\approx \frac{27}{8} BT n_{\text{head}} d$.

Denote the memory requirement of KV cache in Baseline-XL by M_{KV} , and UQ-KV cache in CASTLE(-SWL) by $M_{\text{UQ-KV}}$. Then, we have

- $M_{\text{UQ-KV}} \leq \frac{9}{8} M_{\text{KV}}$ for any t .
- $M_{\text{UQ-KV}} \approx \frac{27}{32} M_{\text{KV}}$ when $t \gg W$ (W is the window size of CASTLE-SWL).

Next, we illustrate the concrete memory requirements of the KV cache (Baseline-XL) and the UQ–KV cache (CASTLE(-SWL)-XL) at different sequence lengths. Table 30 reports the memory usage for a single layer. Notably, CASTLE-SWL-XL already achieves lower memory consumption than Baseline-XL at a sequence length of 1K.

Table 30. Memory usage (in MB) of the KV cache in Baseline-XL and the UQ–KV cache in CASTLE(-SWL)-XL for a single layer at different sequence lengths.

	1K	4K	16K
Baseline-XL	67.11	268.44	1073.74
CASTLE-SWL-XL	66.06	235.93	915.41
CASTLE-XL	75.50	301.99	1207.96

F.3. Prefilling Algorithm

In prefilling stage, we only need to compute U^t . Due to the similarity between (2) and standard causal attention, we can implement prefilling by FlashAttention-like kernels. Its pseudocode is in Algorithm 5.

Algorithm 5 Prefilling Algorithm

Require: $Q^U, K^U, V^U, Q^C, K^C, V^C \in \mathbb{R}^{L \times d}$

Ensure: Output $O \in \mathbb{R}^{L \times d}$

- 1: **UQ–KV cache construction:**
 - 2: Initialize $U = \mathbf{0}^{L \times d}$
 - 3: **for** $k = 0, \dots, N - 1$ **in parallel do**
 - 4: Load $Q_{T_k,:}^U$ from HBM to on-chip SRAM
 - 5: **for** $j = k, \dots, N - 1$ **do**
 - 6: Load $K_{T_j,:}^U, V_{T_j,:}^U, U_{T_k,:}$ from HBM to SRAM
 - 7: Compute $A_{T_k,T_j}^U = \sigma \left(\frac{Q_{T_k,:}^U (K_{T_j,:}^U)^\top}{\sqrt{d}} + M_{T_k,T_j}^U \right)$
 - 8: Update $U_{T_k,:} \leftarrow U_{T_k,:} + A_{T_k,T_j}^U V_{T_j,:}^U$
 - 9: Write $U_{T_k,:}$ back to HBM
 - 10: **end for**
 - 11: **end for**
 - 12: Save UQ–KV cache $U, Q^U, K^C, V^C \in \mathbb{R}^{L \times d}$
 - 13: Call Algorithm 1 to compute O
 - 14: **return** O
-

G. Additional Illustrations in the Definition of CASTLE

In this section, we give additional illustrations of CASTLE and CASTLE-SWL.

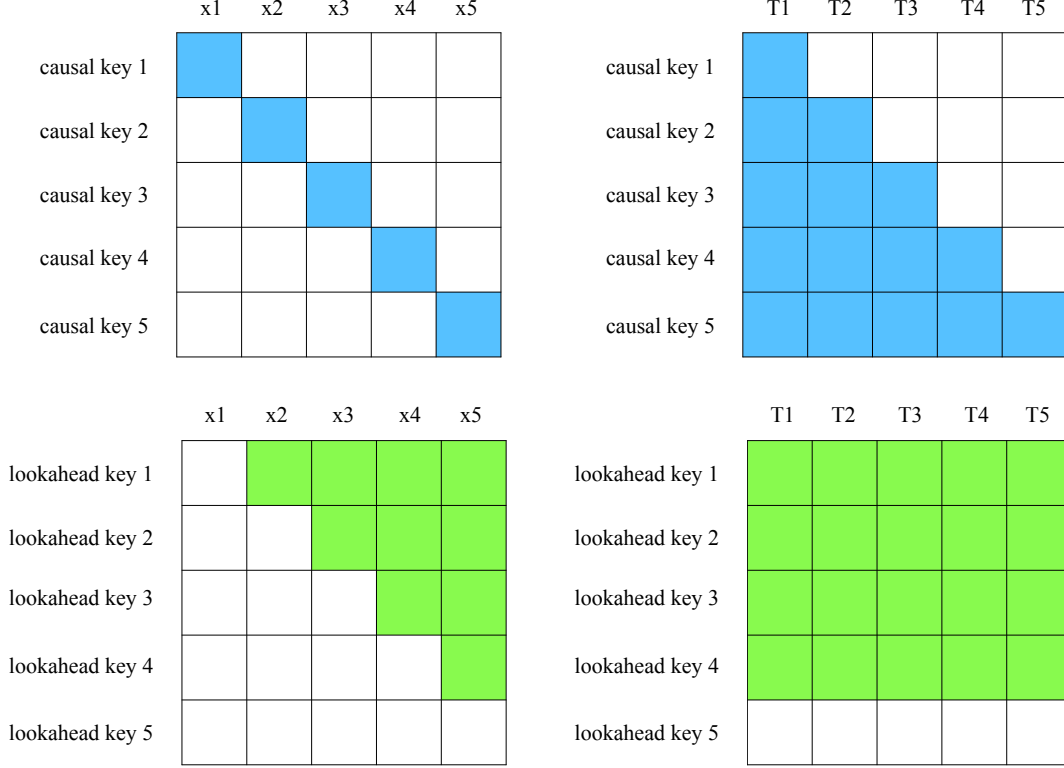


Figure 13. Receptive fields of causal keys and lookahead keys with respect to contextualized representations and tokens (excluding the first layer) when generating the 6th token. Tokens are denoted by T_i and their contextualized representations by x_i . Causal key i corresponds to the causal key of token i , while lookahead key i corresponds to the lookahead key of token i . When generating the $(t + 1)$ -th token, for token s ($s < t + 1$), the causal key of token s is a projection of x_s . Due to the softmax in attention, except in the first layer, causal keys of s encode information from tokens $\{T_1, \dots, T_s\}$. For token $s < t$, lookahead keys of s incorporate information from $\{x_{s+1}, \dots, x_t\}$ and attend over all existing tokens $\{T_1, \dots, T_t\}$. Since the last row of M^U is defined as $[M_t^U]_{t,:} = (-\infty)^{1 \times t}$, lookahead keys of token t are all-zeros vectors and thus have empty receptive fields when generating the $(t + 1)$ -th token.

$$\mathbf{M}_t^U \in \mathbb{R}^{t \times t}$$

$-\infty$	0	0	0	$-\infty$	$-\infty$
$-\infty$	$-\infty$	0	0	0	$-\infty$
$-\infty$	$-\infty$	$-\infty$	0	0	0
$-\infty$	$-\infty$	$-\infty$	$-\infty$	0	0
$-\infty$	$-\infty$	$-\infty$	$-\infty$	$-\infty$	0
$-\infty$	$-\infty$	$-\infty$	$-\infty$	$-\infty$	$-\infty$

Figure 14. CASTLE-SWL differs with CASTLE only in the definition of $\mathbf{M}_t^U$. This figure shows an illustration of $\mathbf{M}_t^U$ in the lookahead keys of CASTLE-SWL, as defined in (8), when generating the $(t+1)$ -th token. Each lookahead key of token s ($s < t$) has access to representations $\{\mathbf{x}_k : s+1 \leq k \leq \min\{t, s+W\}\}$, i.e., up to W subsequent tokens. Also, no information from tokens which are not yet generated is leaked and thereby preserving the autoregressive property. In this figure, we set $t = 6$ and window size $W = 3$. For example, lookahead keys of token 2 can “see” $\{\mathbf{x}_3, \mathbf{x}_4, \mathbf{x}_5\}$, while lookahead keys of token 4 can “see” $\{\mathbf{x}_5, \mathbf{x}_6\}$. As analyzed in Section 1.4 and 1.5, CASTLE-SWL has the same complexities with CASTLE in both training and inference, however, it can further reduce FLOPs and improves efficiency in practice through the use of sliding windows.